National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering

Individual Final Project

Moderated Learning Management System

Software Design Document

Student Name: MAKHOTSO RAMOCHELA

Student Number: 202321185

1. Introduction

1.1 Purpose and scope

This document is the software design of the Moderated Learning Management System (LMS). It covers the deliverable of Phase 3. It contains the four parts that the assignment brief asks for in Phase 3: the architecture (Section 2), the class design (Section 3), the behavioral models (Section 4) and the database design (Section 5). Section 6 is the requirements traceability matrix with its design elements filled in, as the brief requires. Appendix A is the SQL schema.

2. Architecture

2.1 The chosen style: layered architecture

The system uses a layered architecture (Buschmann et al., 1996; Sommerville, 2016). The program is split into four layers. Each layer has one job, and a layer uses only the layer below it. The style was chosen because it gives a separate place to the console, the business rules, the data access and the outside tools. Table 2.1 describes the layers.

Table 2.1: The four layers

Layer	Its job	Main parts	What it must not do
1. Presentation	Show text on the screen, read what the user types, and call the services.	Console menus (one for each role and one for log in), InputReader, Application (main)	It must not hold business rules and must not contain SQL.
2. Business logic	Hold the rules of the system: permissions, late penalty, status changes, moderation and audit entries.	Eight services, the domain model, the penalty rule (PenaltyStrategy), the interfaces (ports) and the exceptions	It must not print or read the keyboard. It must not include SQLite or libsodium headers. It must not read the clock directly.
3. Data access and infrastructure	Save and load data, and connect the interfaces of Layer 2 to the real tools.	SQLite repositories, Database and Transaction, SystemClock, SodiumPasswordHasher	It must not hold business rules and must not call a service.
4. Platform	The tools that are outside our own code.	SQLite library, libsodium, the operating system (clock and files), the file lms.db	Not applicable. It is not code written in this project, apart from the data file lms.db.

The class Application (main) is drawn in Layer 1 only because it is where the program starts. It is the one class that knows all the others: it creates the objects and joins them. In a strict layered style the business layer would call the data layer. Here the business layer owns interfaces (for example IUserRepository and Clock) and Layer 3 implements them, so the business layer never depends on SQLite or libsodium. This is the Dependency Inversion Principle (Martin, 2003). It lets the business layer be tested with fake repositories and a fake clock (NFR-10).

2.2 Rules of the architecture

Table 2.2 gives the rules that the code must follow. A code review and the static analyzer check them in Phase 4.

Table 2.2: Architecture rules

ID	Rule	Reason	Supports
AR-1	Menus call only the services and InputReader. A menu never calls a repository or SQLite.	Keeps the rules out of the menus.	FR-02, NFR-10
AR-2	Services reach data only through the interfaces declared in Layer 2. No file of Layer 2 includes sqlite3.h or sodium.h.	Layer 2 can be tested alone. SQLite can be replaced by changing Layer 3 only.	Brief, NFR-10
AR-3	All SQL is written only in the SQLite repositories and the Database class. Every query is parameterised (values go in ? placeholders). SQL is never built by joining typed text into a string.	Stops SQL injection and keeps all SQL in one place.	Brief, NFR-04

ID	Rule	Reason	Supports
AR-4	Every service method first checks the role of the user and the right of the user to the data (own course, own department, own work). If the check fails it throws PermissionError. The menus also hide options, but the service check is the one that counts.	A menu can be bypassed by a mistake in the menu code. A service cannot.	FR-02, NFR-04, BR-10
AR-5	A service method that changes data does all its changes in one Transaction. The transaction is rolled back automatically if any exception happens before it is committed. A saved mark, its status change and its audit entry are in the same transaction.	A failed action leaves the data as it was. A mark can never be saved without its audit entry.	NFR-08, FR-14, NFR-09
AR-6	Errors are reported by exceptions, using one family of classes (LmsError). Only the menu loop in Layer 1 catches them. It shows a message and returns to the menu.	The program does not stop with an error for a wrong input or a database error.	NFR-08
AR-7	Services read the time only through the Clock interface.	The tests can set any time, so the late penalty bands of BR-01 can be tested at their boundaries.	BR-01, A4, NFR-10
AR-8	Only Application (main) creates the objects and joins them. Each object receives what it needs through its constructor.	No class hides a dependency. The tests can give any class fake parts.	NFR-10
AR-9	The audit interface (IAuditRepository) has only an operation to add an entry and operations to read entries. It has no operation to change or delete an entry.	The program has no way to edit or delete an audit entry. (Section 5 adds a second barrier in the database.)	NFR-09
AR-10	A plain-text password is handled only in AuthService and AdminService, passed straight to PasswordHasher, and is never saved or printed.	Only a salted hash is stored.	NFR-03

2.3 Component diagram

Figure 2.1 is the component diagram. The grey bands are the four layers and the boxes are components. A dashed arrow means "uses". A dashed arrow with a hollow head means that a component implements an interface. These arrows point up from Layer 3 to the interfaces of Layer 2 (Section 2.1). The eight services are shown as one group to keep the diagram readable, and the Exceptions component is used by all layers, so it has no arrows.

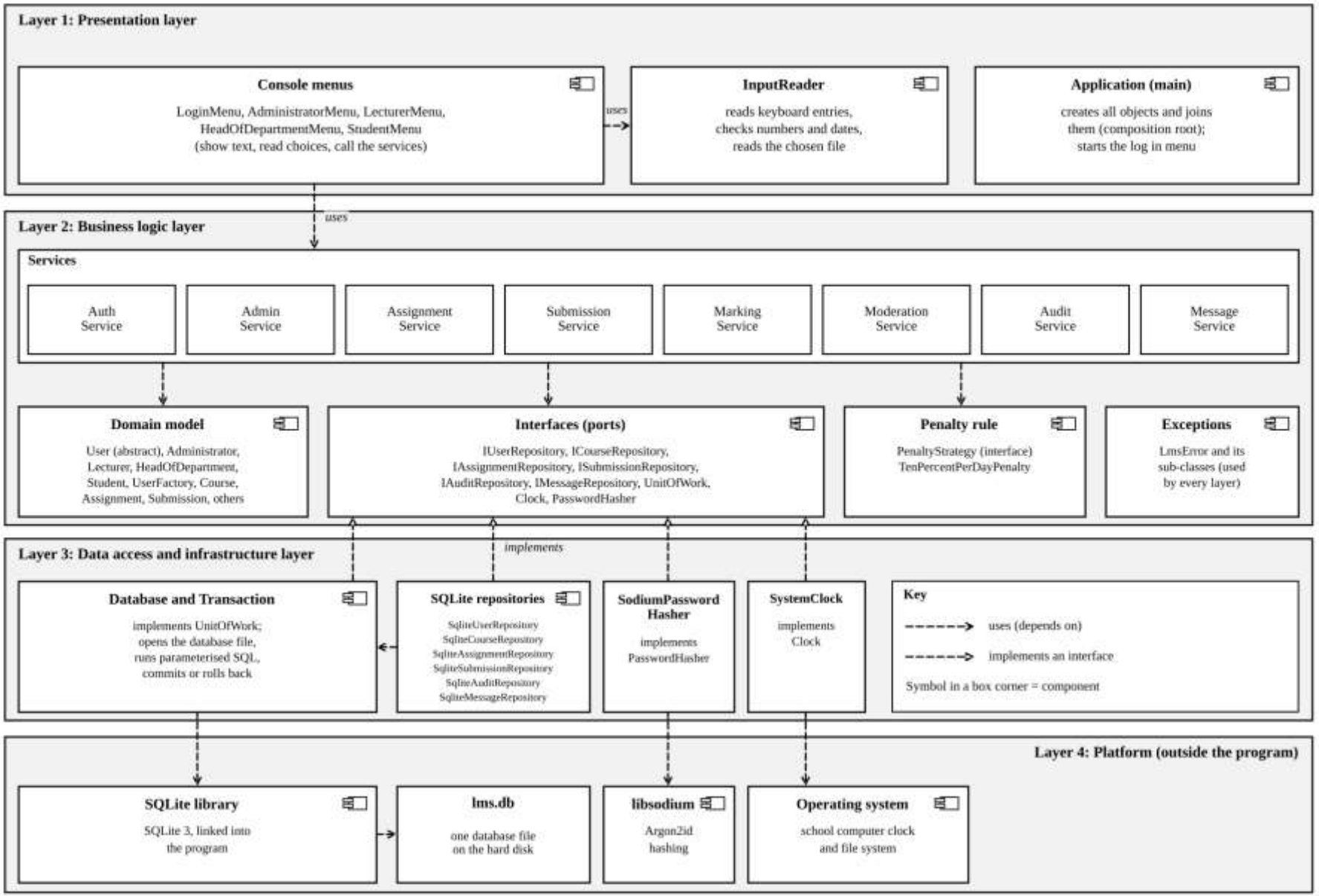


Figure 2.1: Component diagram of the Moderated LMS

2.4 Design rationale

2.4.1 Why layered

Layered architecture was chosen for five reasons. Each one comes from a requirement.

- **The brief asks for a data-access layer.** A layered design has one by definition, and all SQL has one home.
- **NFR-10 asks for coverage of the business-logic layer.** The coverage figure needs a clear border around that layer.
- **The rules change more often than the tools.** The penalty, the status changes and the permissions were the points of dispute in Phase 2 (C1 to C14). They are all in Layer 2, so a change to a rule does not touch the menus or the SQL.
- **Permissions are checked in one place.** Every action goes through a service, so the rules of Table 10 are checked in one layer. This makes NFR-04 easier to test.
- **It is small enough for one person** to design, build and test in the time of Phase 4.

2.4.2 Alternatives that were considered

Two other architectures were considered: Model-View-Controller (MVC), which the brief names, and a client-server design with a database server. Table 2.3 compares the three. "Yes" means that the style meets the need without extra work, "Partly" means that it needs extra work or an added rule, and "No" means that it works against the need.

Table 2.3: Comparison of the three architectures

Need	Layered (chosen)	MVC	Client-server with a database server
Works offline on one computer with no network (IF-04, NFR-05, A2)	Yes	Yes	Partly. It can run on the same computer, but then the server is one more program to start and look after.
Simple to install in 30 minutes by a school technician (NFR-11, IF-03)	Yes. One program and one data file.	Yes	No. A database server must be installed, set up and given accounts.
A separate business-logic layer that can be tested alone (NFR-10)	Yes	Partly. The rules end up in the controllers or in the model, which also shows data to the view.	Partly. It depends on how the client is built.
A clear data-access layer with all SQL in one place (brief)	Yes	Partly. MVC does not say where the data access goes. A rule would have to be added.	Yes
Permission checks in one place (NFR-04)	Yes. In the services.	Partly. In the controllers, which are tied to the menus.	Partly. A server can check, but the rules would be split between server and client.

Need	Layered (chosen)	MVC	Client-server with a database server
Needed by a console program with no screen that updates itself (IF-01)	Yes	No. Views that update when the model changes are not needed.	Not relevant
Can be built and tested by one person in Phase 4	Yes	Yes	No. Two programs and the connection between them.

2.4.3 Rejected alternative 1: Model-View-Controller

In MVC the model holds the data, the view shows it, and the controller handles the input (Krasner and Pope, 1988). It suits programs with windows, where several views must change when the data changes. It was rejected for three reasons.

- **There is nothing for the view to watch.** The program is a console menu. It prints a result after each action and shows the menu again, so the main benefit of MVC is not used.
- **The rules have no clear home.** The penalty, the status changes and the permission checks would end up in the controllers, tied to the console and hard to test alone, or in a model that also knows the database. NFR-10 and the brief need a separate business layer and a data-access layer.
- **The permission checks would be spread over one controller for each menu,** which makes NFR-04 harder to prove.

The idea behind MVC, to keep the screen apart from the rules, is kept. The menus are the view and the controller, and the model is split into a business-logic layer and a data-access layer.

2.4.4 Rejected alternative 2: client-server with a database server

Here a database server such as PostgreSQL would hold the data and the LMS would connect to it as a client. This was rejected for three reasons.

- **It does not match the requirements.** IF-03 says the system needs no other software. A2 and C9 say that version 1.0 runs on one computer and does not share the database between computers, so the main benefit of a server is not needed.
- **It makes installation harder.** NFR-11 needs a technician to install and start the system in 30 minutes. A server adds an installation, an account for the program, a port and a backup plan.
- **It adds risk and work for one developer.** Two programs and the connection between them must be built and kept running on old computers with 2 GB of memory.

The cost is that SQLite cannot be shared safely between computers. All SQL is behind interfaces in Layer 3, so a later version could replace only those classes.

3. Class design

3.1 Reading the class diagrams

The class diagram shows visibility, types, multiplicities, inheritance and interfaces. One diagram of all the classes would be too big to read, so it is drawn as 10 figures, one for each part of the design. Together they are one class diagram. A class that appears only as a box with its name is a reference to its full box in another figure. The classes follow Section 2: the services, the domain model, the interfaces and the penalty rule are in Layer 2; the repositories, SystemClock and SodiumPasswordHasher are in Layer 3; the menus and InputReader are in Layer 1. Table 3.1 explains the conventions that are not part of standard UML (Object Management Group, 2017).

Table 3.1: Conventions used in the class diagrams

Notation	Meaning
name: Type	An attribute or a parameter with its type. Types are C++17 types. The prefix std:: is not written, so string means std::string, vector means std::vector, optional means std::optional and unique_ptr means std::unique_ptr.
optional<T>	The value may be missing. For a mark, for example, a submission that is not marked yet has no mark.
Type&	A reference to another object that is given to the class when it is created (constructor injection). It shows which interfaces and services a class depends on.
Name-only box	A reference to a class that is drawn in full in another figure.
Getters and setters	Simple getters and setters of private attributes (for example getTitle and setTitle) are not drawn, to save space. Operations that contain a rule are drawn. AuditEntry has getters only, and no setters.
Acting user (not drawn)	Every operation of the services in Figures 3.4 to 3.6 has a first parameter actor: User&, the logged-in user. It is not drawn, to save space. The service uses it for the permission check (AR-4 of Section 2) and for the values grantedBy, markedBy and changedBy. In Section 4 it is written as actor in the calls.
Links to users that are not drawn	Some links go from a class to a user and are kept as an id attribute without a line, because the line would cross the diagram: Extension.grantedBy, Submission.markedBy, StatusChange.changedBy and AuditEntry.changedBy.

The figures follow on the next pages, one for each page.

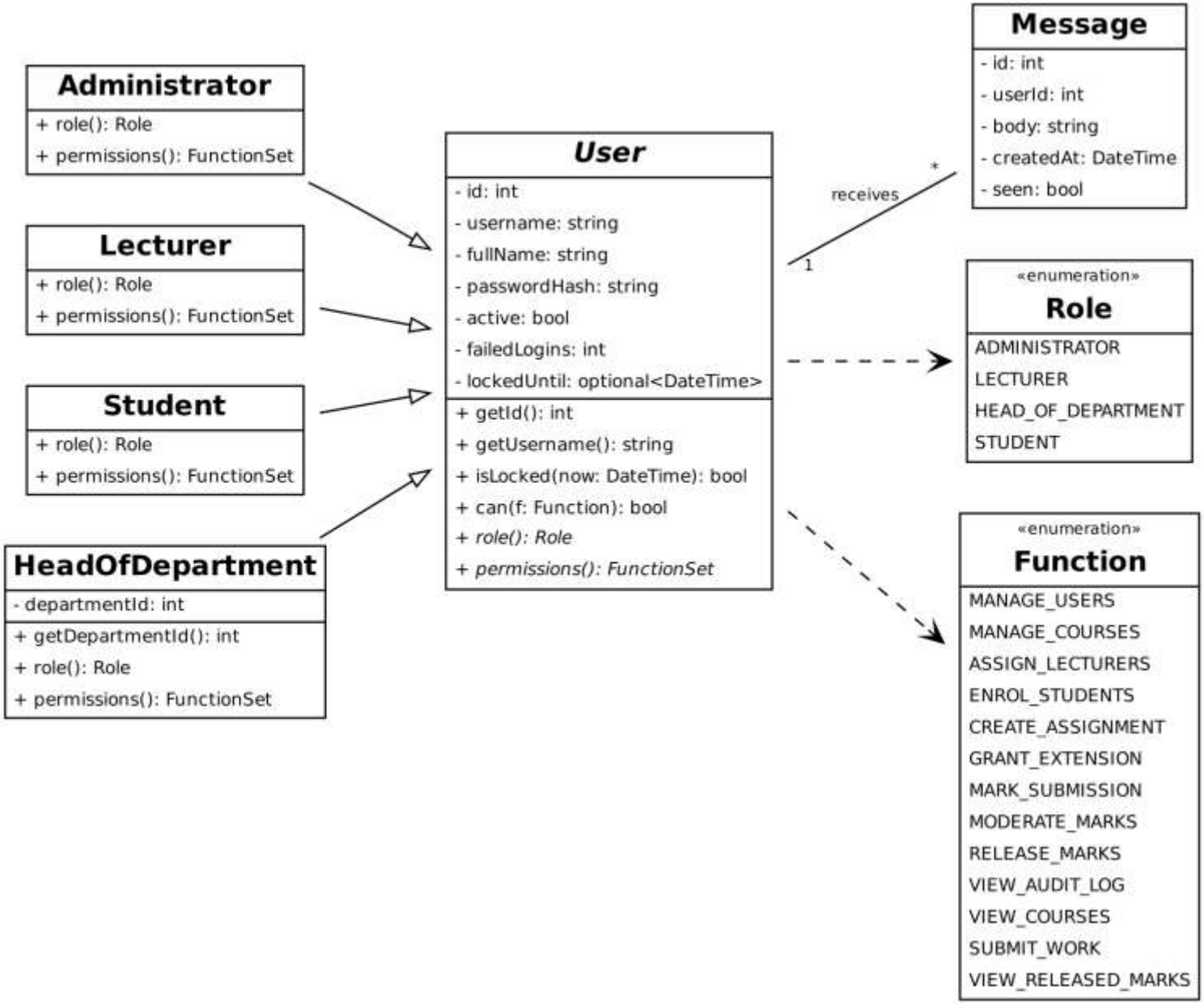


Figure 3.1: Users, roles and messages

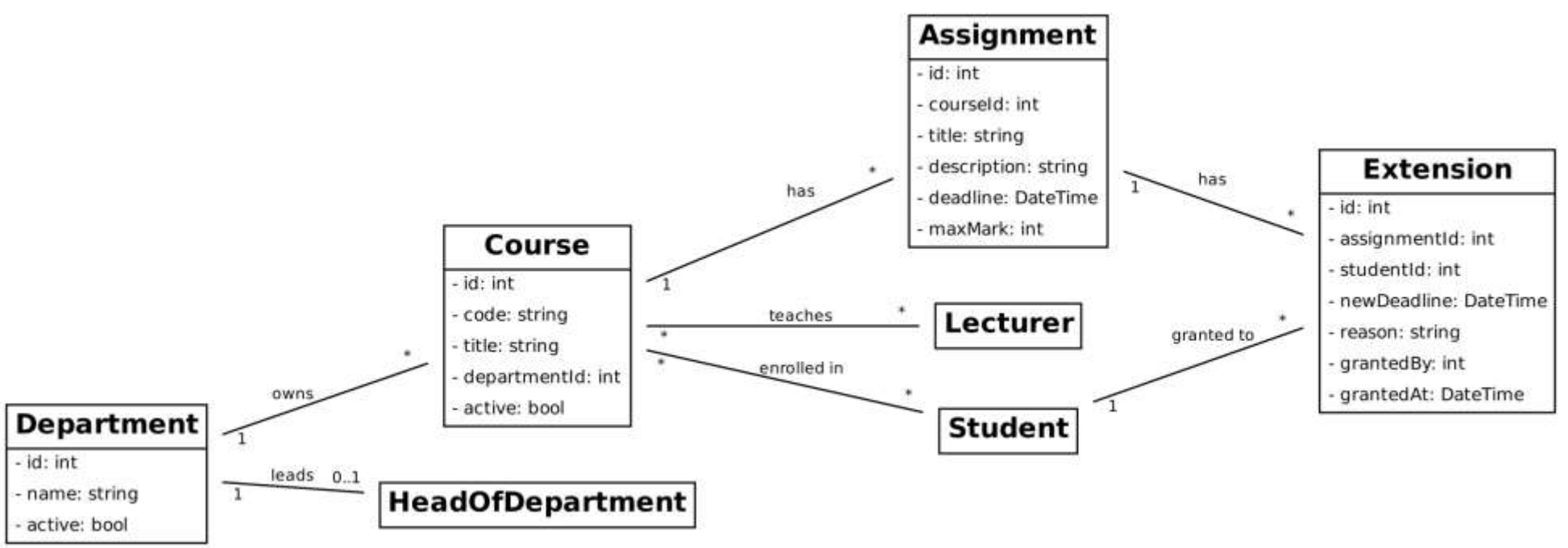


Figure 3.2: Departments, courses, assignments and extensions

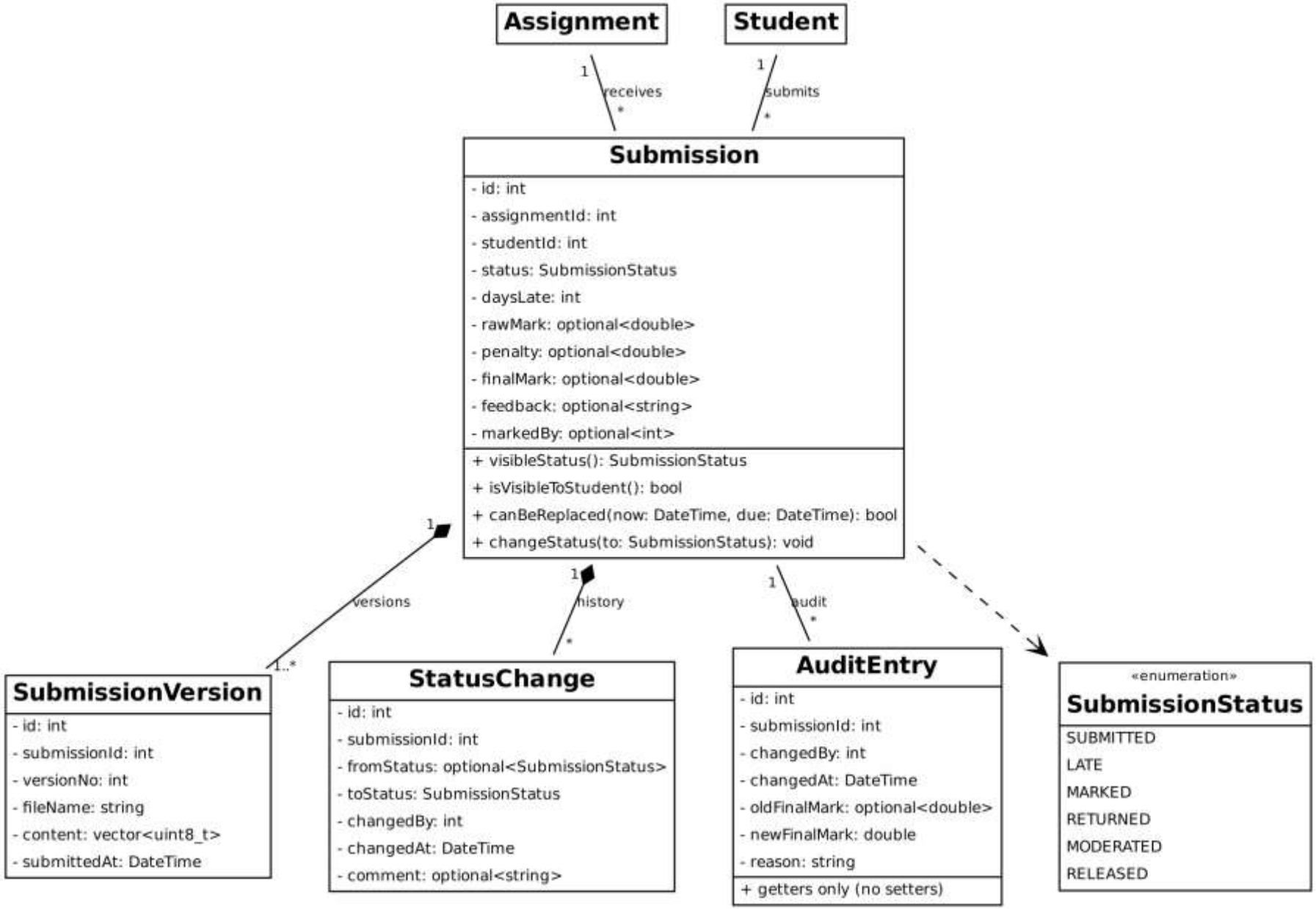


Figure 3.3: Submissions and their records

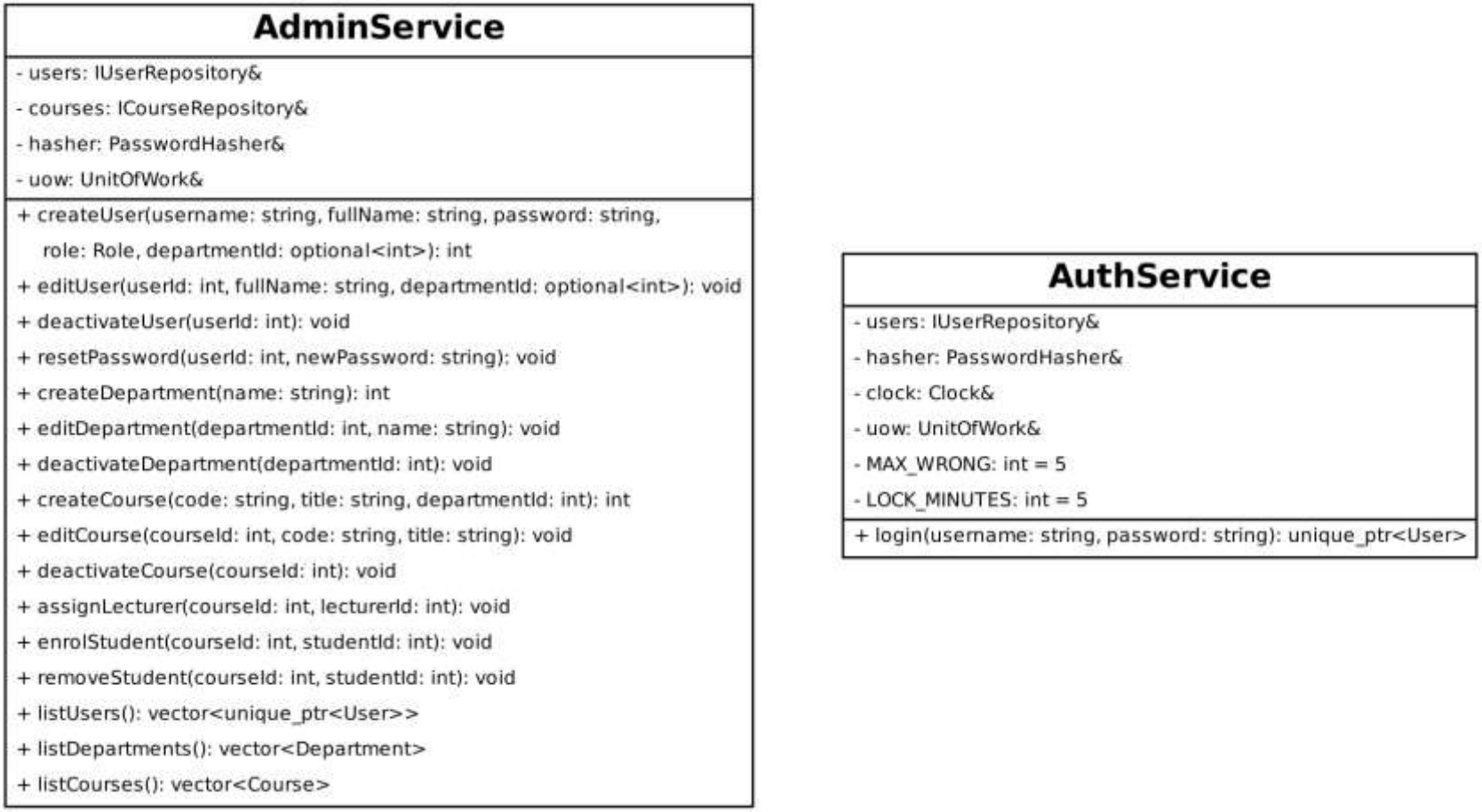


Figure 3.4: Services 1: AuthService and AdminService

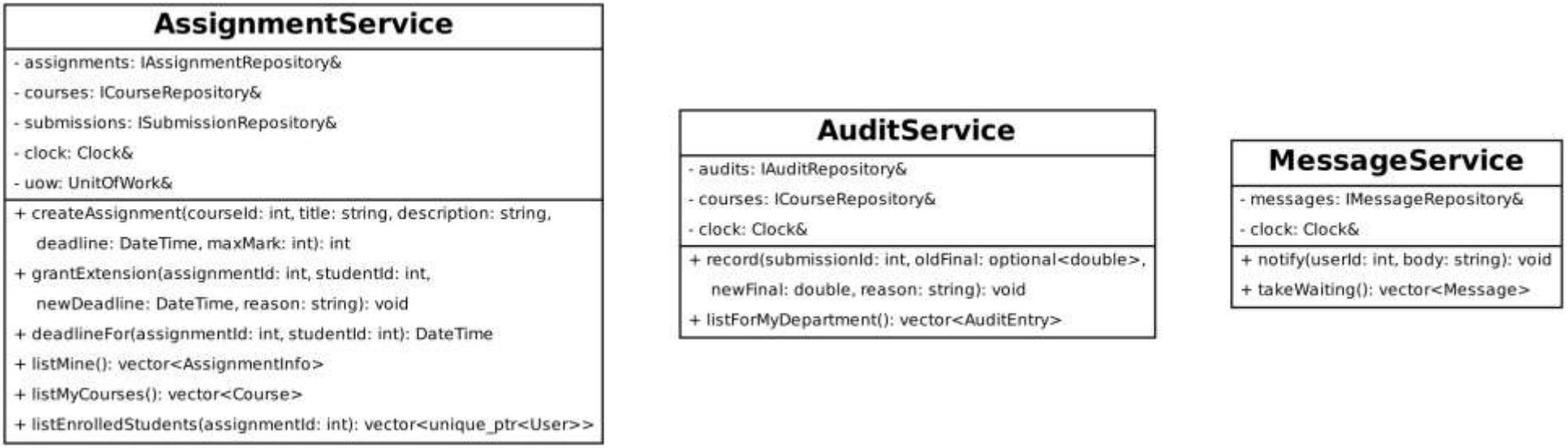


Figure 3.5: Services 2: AssignmentService, AuditService and MessageService

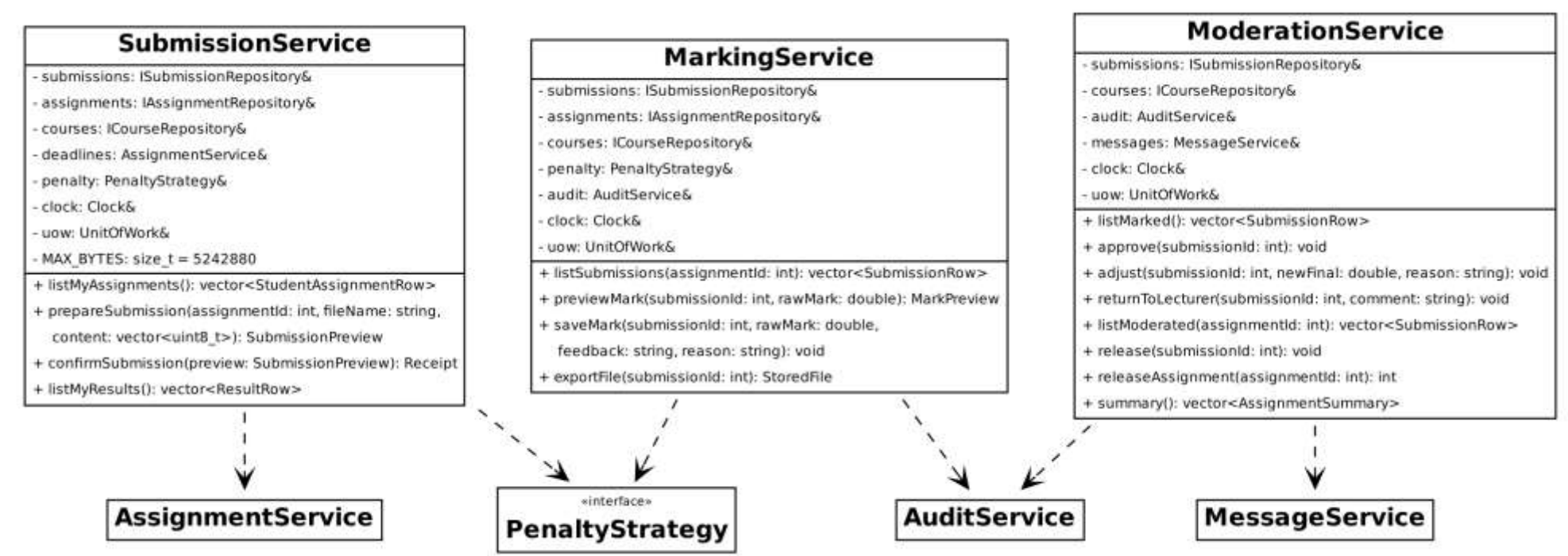


Figure 3.6: Services 3: SubmissionService, MarkingService and ModerationService

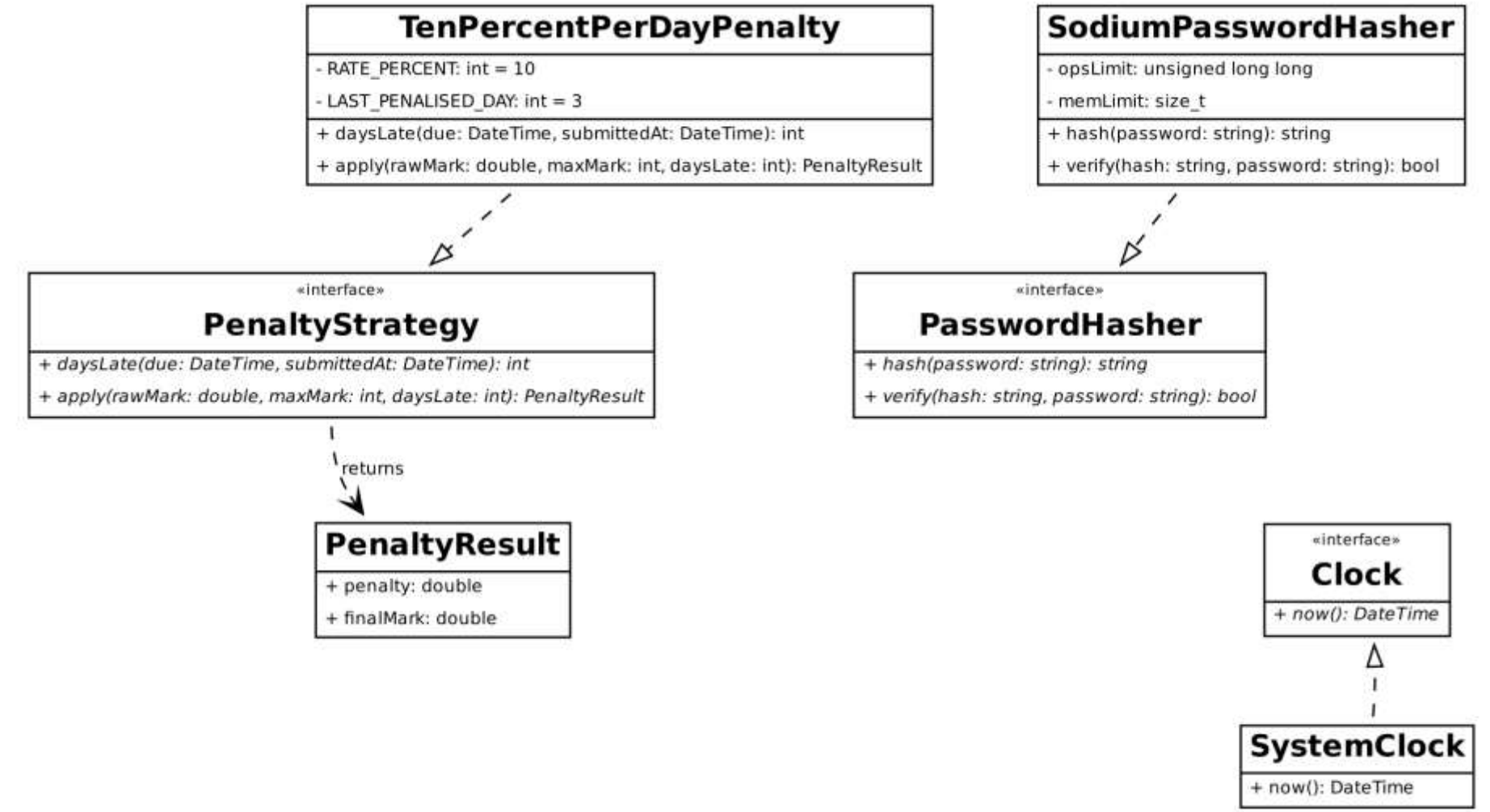


Figure 3.7: Penalty strategy, password hasher and clock

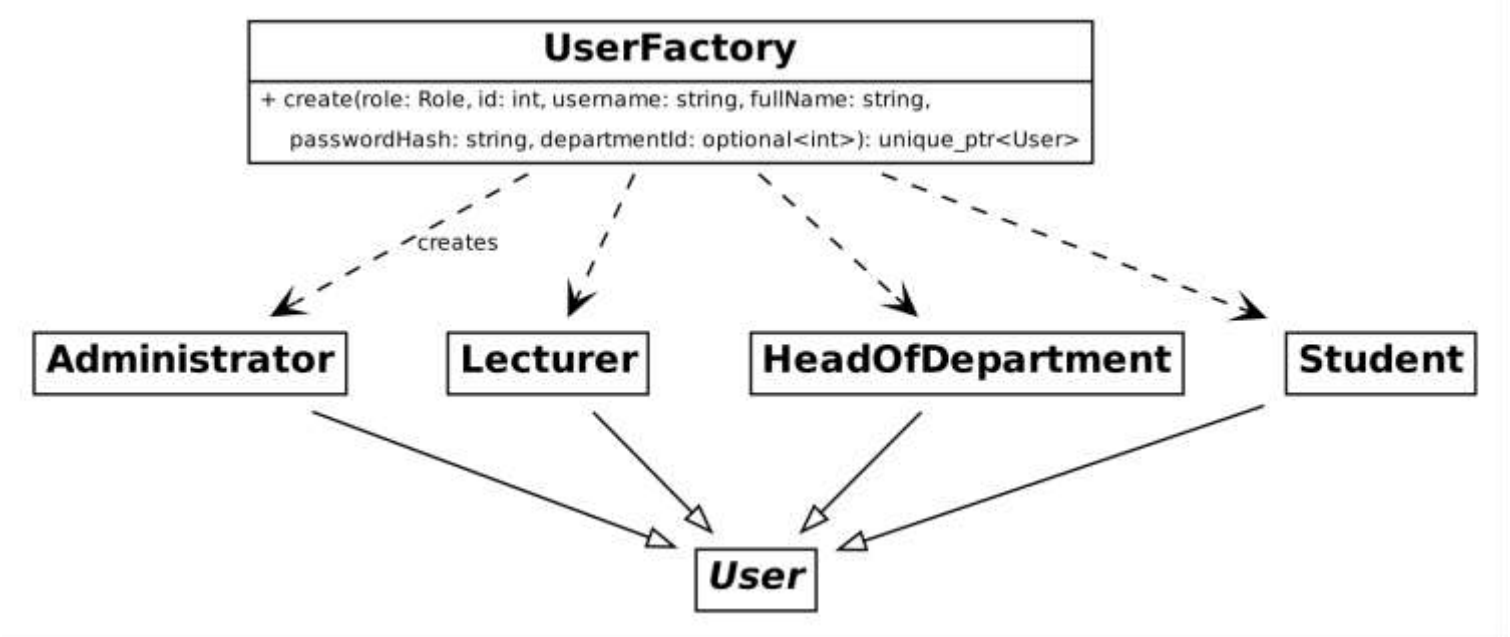


Figure 3.8: Creating users with UserFactory

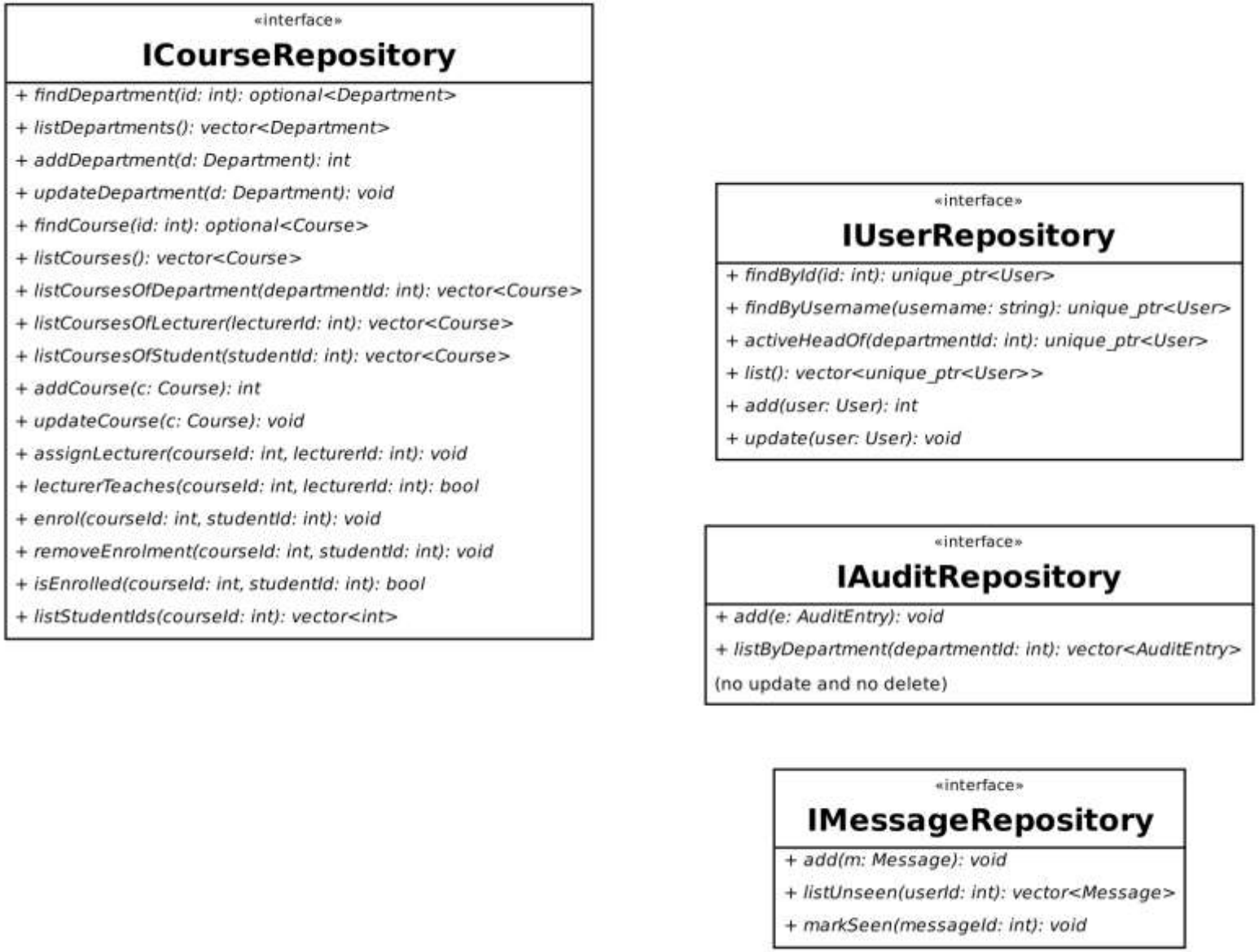


Figure 3.9: Repository interfaces (1 of 2)

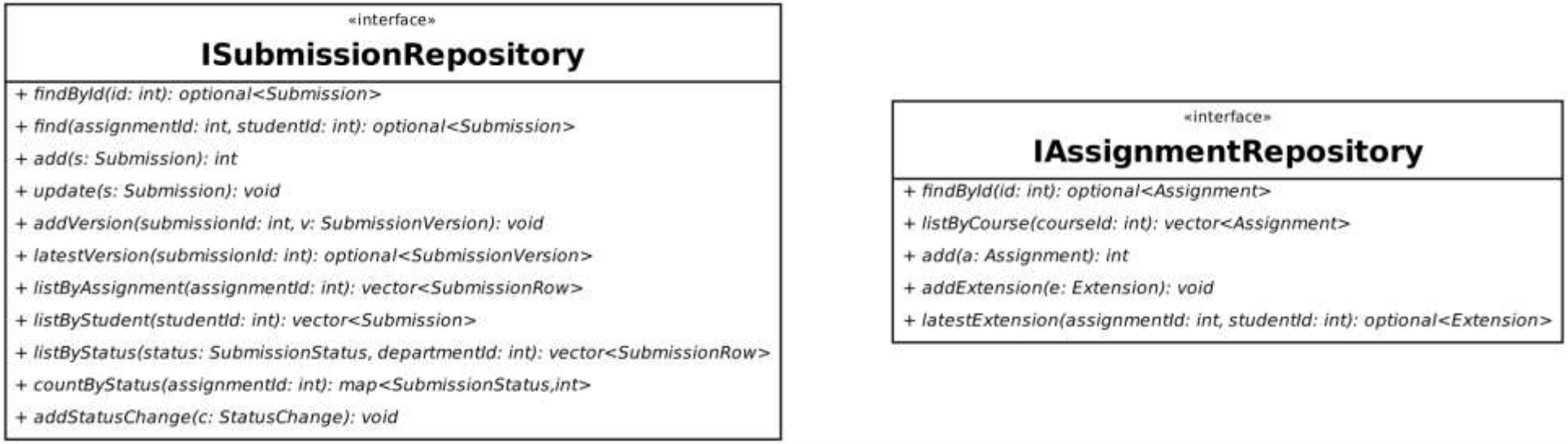


Figure 3.10: Repository interfaces (2 of 2)

3.2 Supporting types

The operations in Figures 3.4 to 3.6 use small data classes that only carry data from a service to a menu. They have public attributes and no rules, so they are not drawn. Table 3.2 lists them. Errors are reported by exceptions. The class LmsError has six sub-classes: ValidationError, PermissionError, NotFoundError, RuleError, DatabaseError and AuthenticationError.

Table 3.2: Supporting types

Type	Fields
DateTime	A date and time to the minute in local time (A4). It has operations to read and write the text form YYYY-MM-DDTHH:MM, to compare two times and to count the minutes between them. This text form is the one saved in the database.
FunctionSet	A set of Function values. It has the operation contains(f).
PenaltyResult	penalty: double (marks taken off); finalMark: double.
AssignmentInfo	assignmentId, courseCode, title, deadline (the deadline of the user who asks, after any extension), maxMark.
StudentAssignmentRow	assignmentId, courseCode, title, deadline (the student's own deadline), maxMark, visibleStatus: optional<SubmissionStatus>. The visible status is empty if the student has not submitted, and it follows BR-02 (only Submitted, Late or Released).
SubmissionPreview	assignmentId, fileName, content, daysLate, penaltyMarks, willGetZero, replacesVersion: optional<int>. It is what the student sees before confirming (FR-17, FR-18). penaltyMarks is the number of marks that will be taken off the maximum mark, and willGetZero is true if the work is 4 or more days late.
Receipt	submissionId, versionNo, fileName, submittedAt, daysLate.
ResultRow	assignmentId, courseCode, title, visibleStatus, and finalMark and feedback. The last two are filled only if the status is Released (FR-19).
SubmissionRow	submissionId, studentName, studentUsername, assignmentTitle, status, submittedAt, daysLate, rawMark, penalty, finalMark, feedback, markedByName. The last five are empty if the submission is not marked.
MarkPreview	rawMark, daysLate, penalty, finalMark. It is what the lecturer sees before confirming (FR-13).
StoredFile	fileName, content: vector<uint8_t>.
AssignmentSummary	assignmentId, courseCode, title, marked, moderated, released (the three counts of FR-25).

3.3 Associations and multiplicities

Associations and multiplicities are drawn in Figures 3.1 to 3.3. Most links are kept as an id attribute (for example Assignment.courseId) and not as a pointer. The ids are the same as the database columns (Section 5), a list of 800 submissions can be loaded without loading all related objects, and there are no pointer cycles. The cost is that a service asks a repository for a related object when it needs it.

3.4 Inheritance and interfaces

Inheritance and interfaces are drawn in the class figures, for example the four user classes in Figure 3.1 and the penalty strategy, password hasher and clock interfaces in Figure 3.7. The brief asks for an abstract User class with polymorphism for the roles. User is abstract and has the abstract operations role() and permissions(). The operation can(f) is written once in User

and calls permissions(), so any user answers "may this user do f?" without an if or a switch on the role. A role does not say which courses or which department a user may use, so the service makes a second check ("own courses", "own department", "own work") with the repositories. Both checks are made at the start of every service operation.

3.5 Design patterns

The design uses six patterns (Gamma et al., 1994; Fowler, 2002). Table 3.3 gives, for each one, where it is used, the problem it solves, why it was chosen and what it replaces. Observer was considered and not used. In this system a message is not sent to a user who is working: messages are stored and shown at the next log in, so nothing waits to be told about an event. A stored Message made by MessageService is simpler. Every service and menu receives the objects it needs through its constructor, and Application is the only class that joins them.

Table 3.3: Design patterns

Pattern	Classes	Problem it solves	Why it was chosen	Rejected instead
Strategy	PenaltyStrategy, TenPercentPerDayPenalty. Used by SubmissionService and MarkingService.	The late-penalty rule (BR-01) was the most argued rule in the case study (C2) and it has special cases (risk R11).	The rule is in one class that can be tested alone at the boundary times of Table 8. Both services use the same object, so they cannot disagree. A new rule is a new class, and only Application changes.	if and else code written in both services, which would put the rule in two places.
Repository (DAO)	Six repository interfaces and six Sqlite classes.	The brief says data is reached only through a data-access layer.	Services work with objects and lists, not with SQL. Tests use fake repositories.	SQL written inside the services.
Factory (simple factory)	UserFactory.create	A user is stored with a role as text, but the program needs an object of the right subclass. This is needed when a user is loaded and when an account is created.	The choice of subclass is made in one place. A new role changes the factory and the enumeration only.	new Administrator, new Lecturer and so on written in several places, each with its own switch.
Adapter	SodiumPasswordHasher adapts the libsodium functions crypto_pwhash_str and crypto_pwhash_str_verify to PasswordHasher.	libsodium is a C library with its own types and settings. The business layer must not include its header (AR-2).	AuthService and AdminService see two simple operations, hash and verify. The tests use a fast fake hasher, because a real Argon2id hash is made to take 100 to 500 ms (NFR-03), which would make a large test run slow.	Calling libsodium directly from the services.
Template Method	Menu::run, with printChoices and handle as the steps that the five menu classes fill in.	Every menu needs the same loop and the same error handling (NFR-08).	The try and catch of LmsError is written once, so no menu can forget it.	A separate loop and try block in each menu.

Pattern	Classes	Problem it solves	Why it was chosen	Rejected instead
Unit of Work (with RAII)	UnitOfWork, Transaction, SqliteTransaction	A mark, its status change and its audit entry must be saved together or not at all (NFR-08, FR-14).	The Transaction rolls back by itself when it is destroyed without commit(), including when an exception leaves the function.	Calls to BEGIN, COMMIT and ROLLBACK written by hand in every service operation, where one could be forgotten.

3.6 SOLID principles

Table 3.4 explains each SOLID principle (Martin, 2003) with the classes that show it. It also names the places where the design does not follow a principle fully, and why.

Table 3.4: SOLID principles in the design

Principle	How the design follows it	Where it is only partly followed
S: Single Responsibility (a class has one reason to change)	PenaltyStrategy changes only if the penalty rule changes. AuditService writes and reads audit entries and nothing else. Each repository holds the SQL of one group of data. The menus only show text and read input, and the services only hold rules. The password hash is made in one class.	AdminService has 16 operations. They all belong to the work of the Administrator (Table 10, first row), so they were kept in one class. If it grows, it can be split into a user service and a course service. ICourseRepository covers four tables that are always used together.
O: Open for extension, closed for change	A new penalty rule is a new PenaltyStrategy class. A new kind of menu is a new subclass of Menu. A new password library is a new PasswordHasher class. None of the services change.	A new role needs a new User subclass, a new value in Role and a new case in UserFactory. The factory is changed, so the design is not fully closed here. The change is in one known place.
L: Liskov Substitution (a child can be used where the parent is expected)	Any User subclass can be used where a User is expected, because the services only use the User operations. TenPercentPerDayPenalty keeps the promise of PenaltyStrategy: daysLate is 0 or more, and finalMark is never below 0. A fake Clock or a fake repository can replace the real one without the service noticing. The subclasses of User do not add conditions to the operations of User.	None found.
I: Interface Segregation (no class is forced to depend on operations it does not use)	There are six small repository interfaces and not one large database interface. AuditService is given only IAuditRepository and ICourseRepository. Clock has one operation and PasswordHasher has two. UnitOfWork and Transaction are separate interfaces.	ICourseRepository and ISubmissionRepository are the largest interfaces (17 and 11 operations). Each is used as a whole by the services that need it.

Principle	How the design follows it	Where it is only partly followed
D: Dependency Inversion (depend on abstractions, not on details)	The services depend on the interfaces of Layer 2 and never on the Sqlite classes, libsodium or the system clock. The Layer 3 classes depend on the interfaces, as the arrows in Figure 2.1 show. Application joins the parts (AR-2, AR-7, AR-8).	The services depend on the concrete classes of the domain model (for example Submission). These are plain data classes with their own rules and no outside tools, so no benefit comes from hiding them behind interfaces. SubmissionService, MarkingService and ModerationService also depend on the concrete classes AssignmentService, AuditService and MessageService. These are inside the same layer.

4. Behavioral models

4.1 State machine: life cycle of a submission

Figure 4.1 shows the life cycle of a Submission. A submission has one status at a time: Submitted, Late, Marked, Returned, Moderated or Released. The black dot is the start. The transitions are numbered T1 to T13, and each label gives the event, the guard in square brackets and the action after the slash. Submission::changeStatus is the only operation that changes a status, and it accepts only the changes drawn in Figure 4.1. Any other change throws a RuleError. The italic line in each state shows what a student sees: only Submitted or Late until the status is Released. An extension for a student who has no submission, or whose submission is Submitted, changes only the student's deadline, so it has no arrow.

4.2 Activity diagram: from submission to released mark

Figure 4.2 shows the whole process from the moment the student chooses a file to the moment the student sees the released mark. Its lanes are Student, System (the program), Lecturer and Head of Department. A box is done by the actor of its lane and a diamond is a decision. The process has three parts.

- **Submit.** The system checks the file and the replace rule, works out the student's own deadline and the days late, and warns about a late penalty. When the student confirms, it saves the version and gives a receipt.
- **Mark.** The lecturer types the raw mark, sees the penalty and the final mark, types the feedback and confirms. The system saves the mark, sets Marked and writes the audit entry.
- **Moderate and release.** The Head of Department approves, adjusts or returns the mark. Approve and Adjust give Moderated. Return gives Returned and a message for the lecturer, who marks again. The Head of Department then releases, the system sets Released and leaves the messages, and the student sees the mark.

4.3 Sequence diagrams

Figures 4.3 to 4.7 show "Submit assignment" (UC-16, with the late path UC-17 and the resubmission path UC-18) and "Moderate and release marks" (UC-12, UC-13 and UC-14). Submit assignment is drawn in three figures that follow each other, because one figure would be too small to read. The lifelines are classes of Section 3, and boxes marked «interface» are interfaces. Every service operation has a first parameter actor, the logged-in user (Table 3.1).

4.3.1 Submit assignment

Figure 4.3. The menu lists the student's assignments with the student's own deadline and the visible status (BR-02). The student chooses one and a file. InputReader reads the file, and prepareSubmission refuses an empty file or a file over 5,242,880 bytes.

Figure 4.4. The service reads the time from Clock, the student's deadline D and the earlier submission. If an earlier submission exists and its status is not Submitted, or the time is later than D, it throws a RuleError (BR-03). Otherwise PenaltyStrategy gives the days late n and, if n is more than 0, the penalty (the mark will be 0 if n is 4 or more, BR-01). The menu shows one of three summaries (on time, replacement, late) and the student confirms or cancels.

Figure 4.5. confirmSubmission repeats the checks with the time now, because the deadline may have passed while the student was reading. If the result is different, the menu asks again. Otherwise the service saves the submission, version 1 and a status change (a replacement saves only the next version) in one transaction and returns the Receipt. A database error rolls everything back and nothing is saved (E4 of UC-16).

4.3.2 Moderate and release marks

Figure 4.6. The Head of Department sees the Marked submissions of the own department (BR-07) and the counts for each assignment (FR-25). Approve changes Marked to Moderated. Adjust checks the status, the range and the reason, then saves the new final mark, writes the audit entry and changes the status in one transaction. Return changes Marked to Returned and leaves a message with the comment for the lecturer.

Figure 4.7. The Head of Department releases all the Moderated submissions of one assignment, or one submission. The service checks that each is Moderated and in the department. In one transaction it changes each to Released and leaves a message for the student and the lecturer. If any step fails, nothing is committed, so no student sees half of a release.

In the models, UC-12 and UC-16 differ from their Phase 2 text in two points. The design shows one list of the assignments of all the user's courses and not a course first and then an assignment (this gives fewer steps, NFR-06). It also asks the student to confirm once, after the file is chosen and checked, and always before saving. UC-16 asks for confirmation only for late work and for a replacement, and at step 2 of A3.

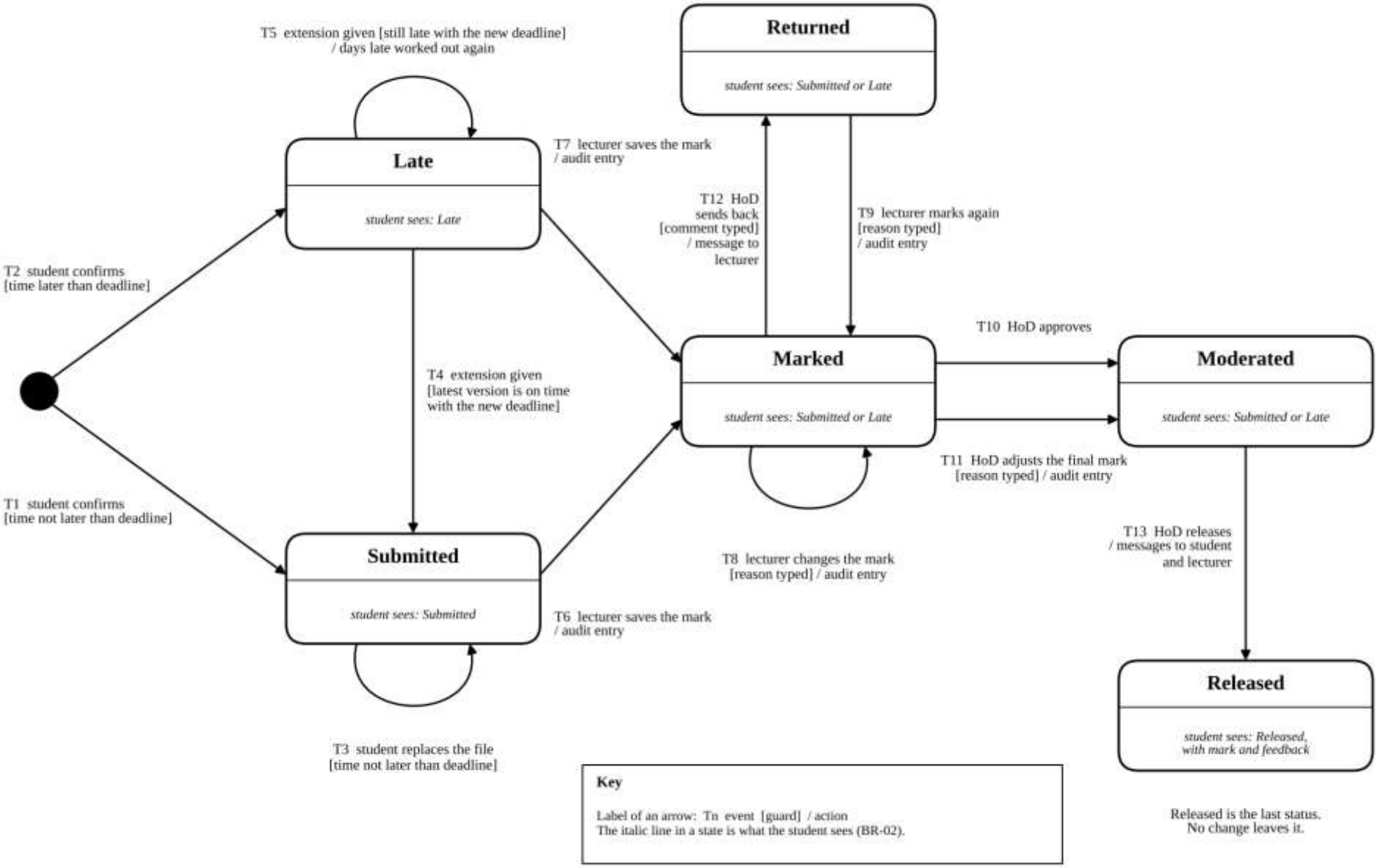


Figure 4.1: Life cycle of a submission (state machine)

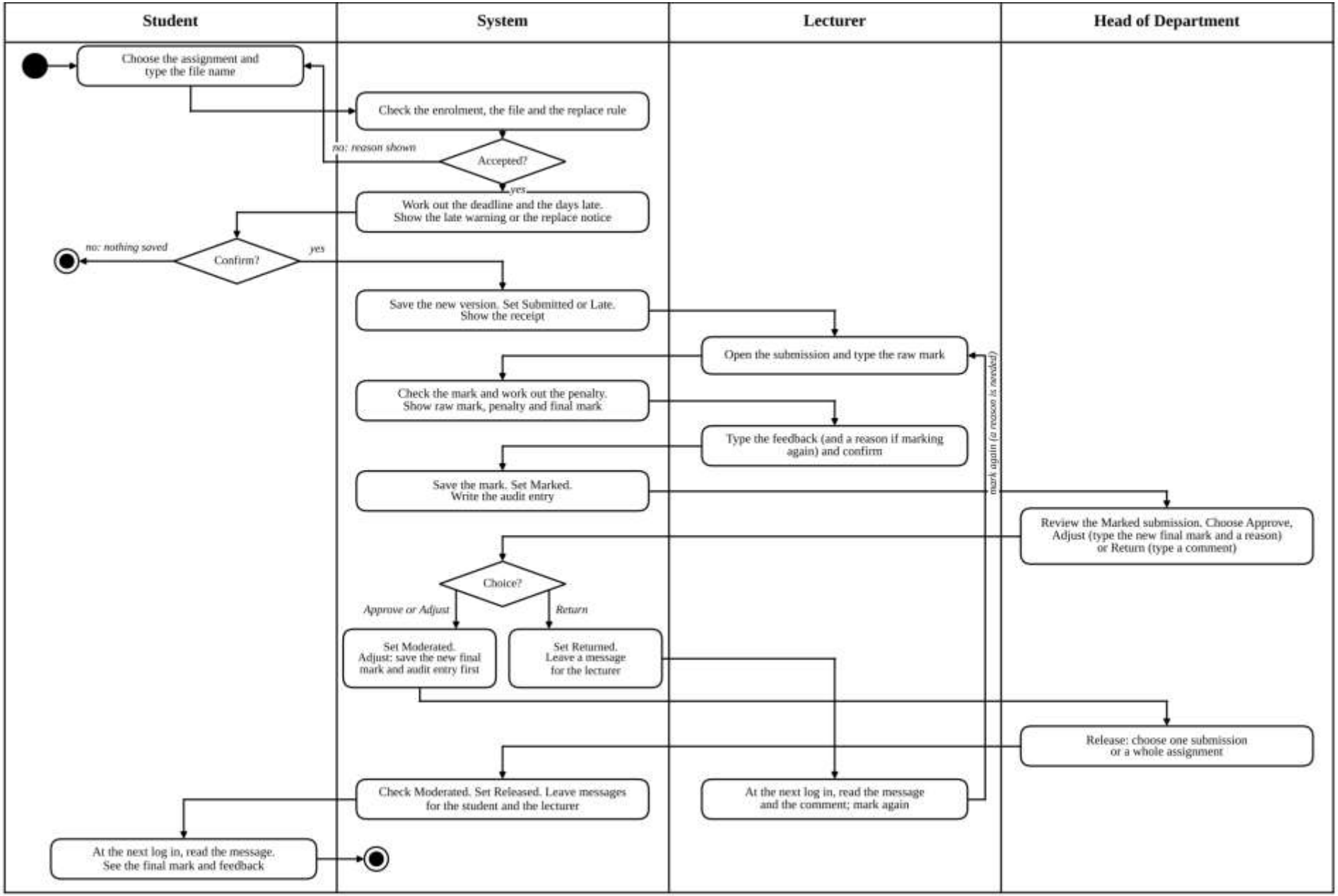


Figure 4.2: From submission to released mark (activity diagram with swimlanes)

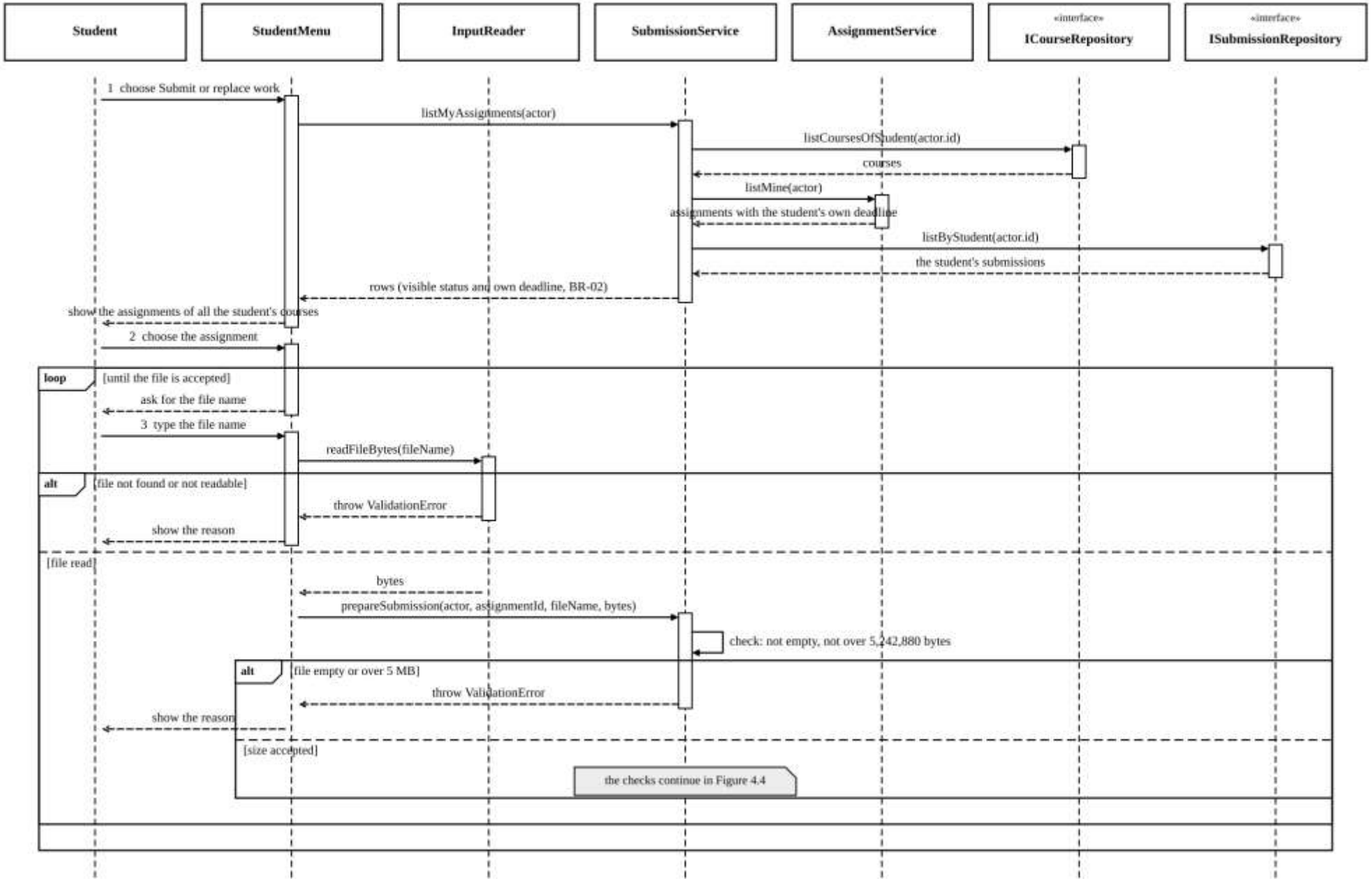


Figure 4.3: Submit assignment (1 of 3): choose the assignment and the file

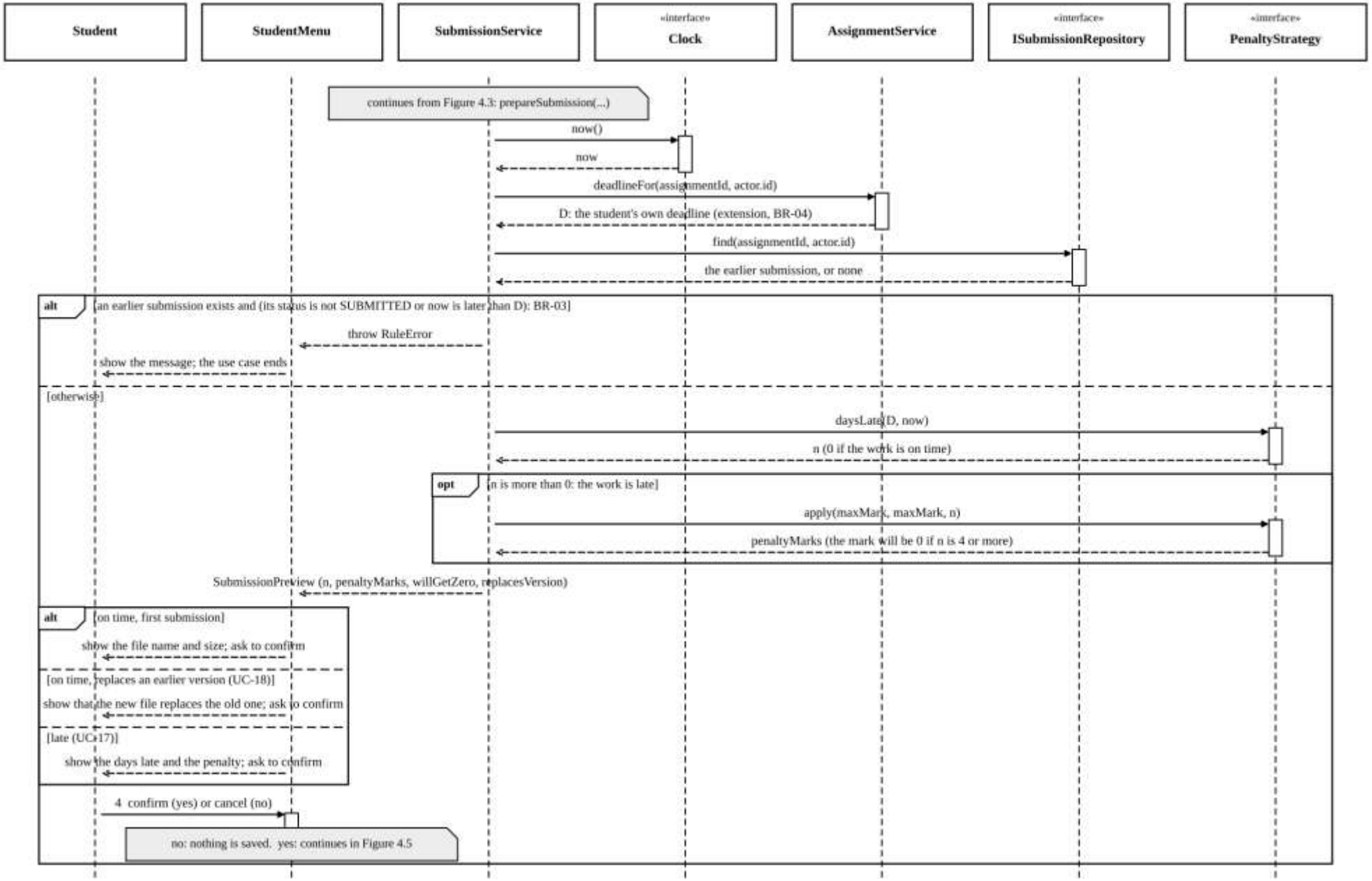


Figure 4.4: Submit assignment (2 of 3): deadline, late path and resubmission path



Figure 4.5: Submit assignment (3 of 3): confirm and save, with the failure path

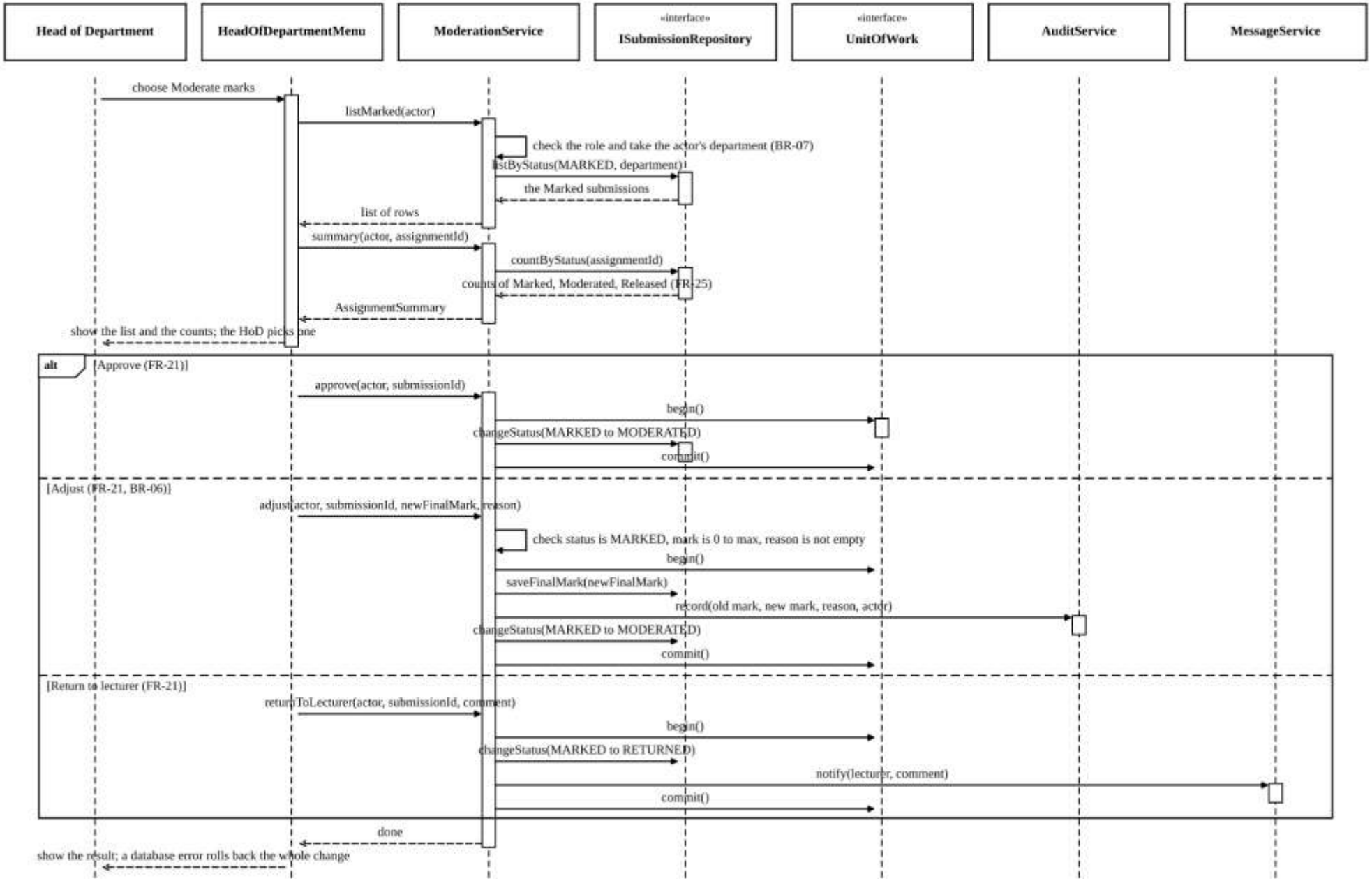


Figure 4.6: Moderate marks: list, approve, adjust and return

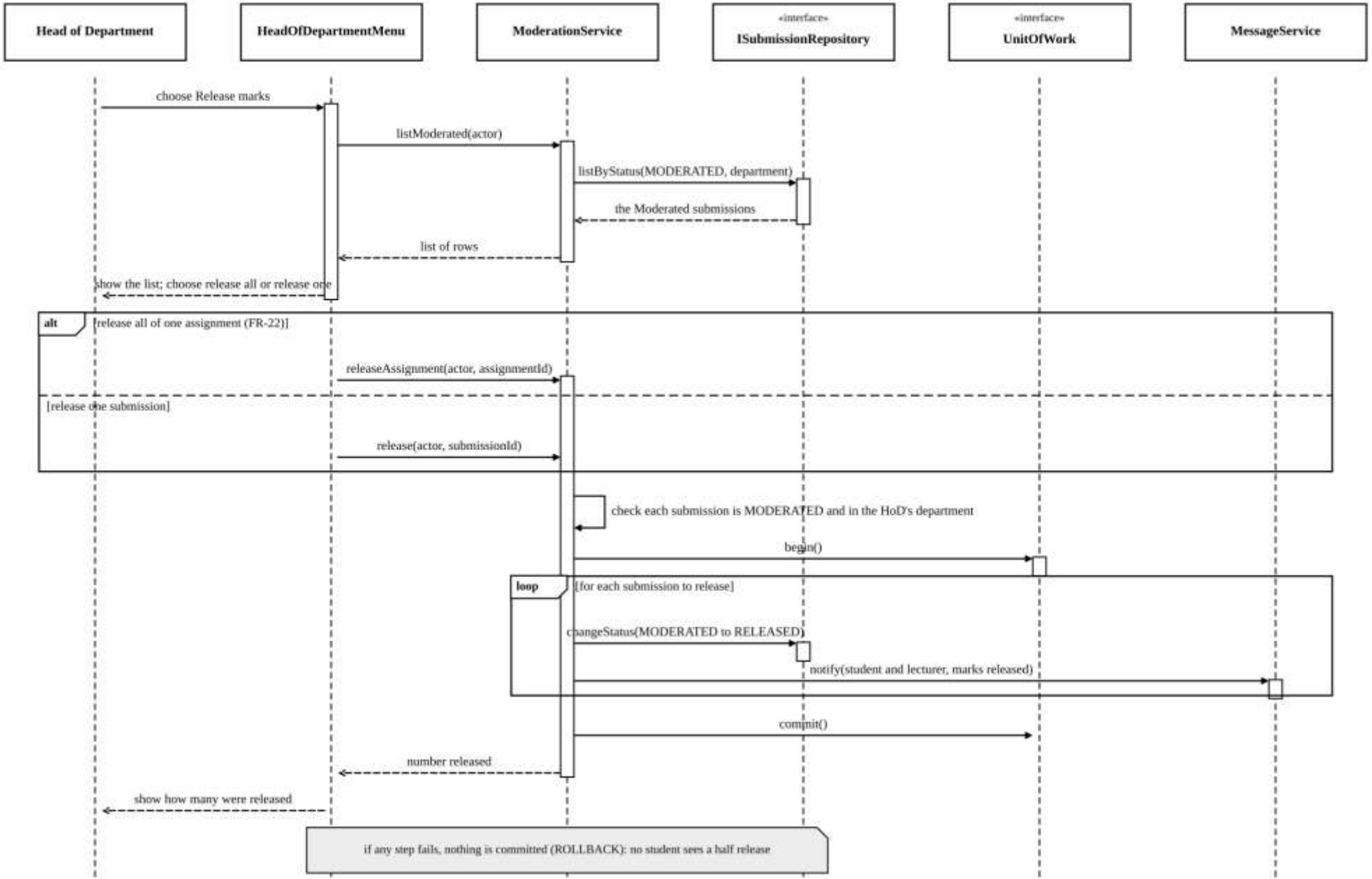


Figure 4.7: Release marks

5. Database design

5.1 Choices for the whole schema

The database is SQLite, one file on the school computer (NFR-05). Only the Sqlite repositories read or write it (AR-3). Dates and times are stored as text in the form YYYY-MM-DDTHH:MM, and yes/no values as 0 or 1. Every link between tables is a FOREIGN KEY, and the program switches foreign keys on each time it opens the database. Users, departments and courses are deactivated and not deleted, so old marks and audit entries always have a row to point to.

5.2 Entity-relationship diagram

Figure 5.1 is the ERD with 12 tables. PK marks a primary key (underlined) and FK marks a foreign key. The lines use crow's foot notation: a bar means exactly one, a circle means zero, and a crow's foot means many. A many-to-many association of the class diagram (course and lecturer, course and student) becomes a link table.

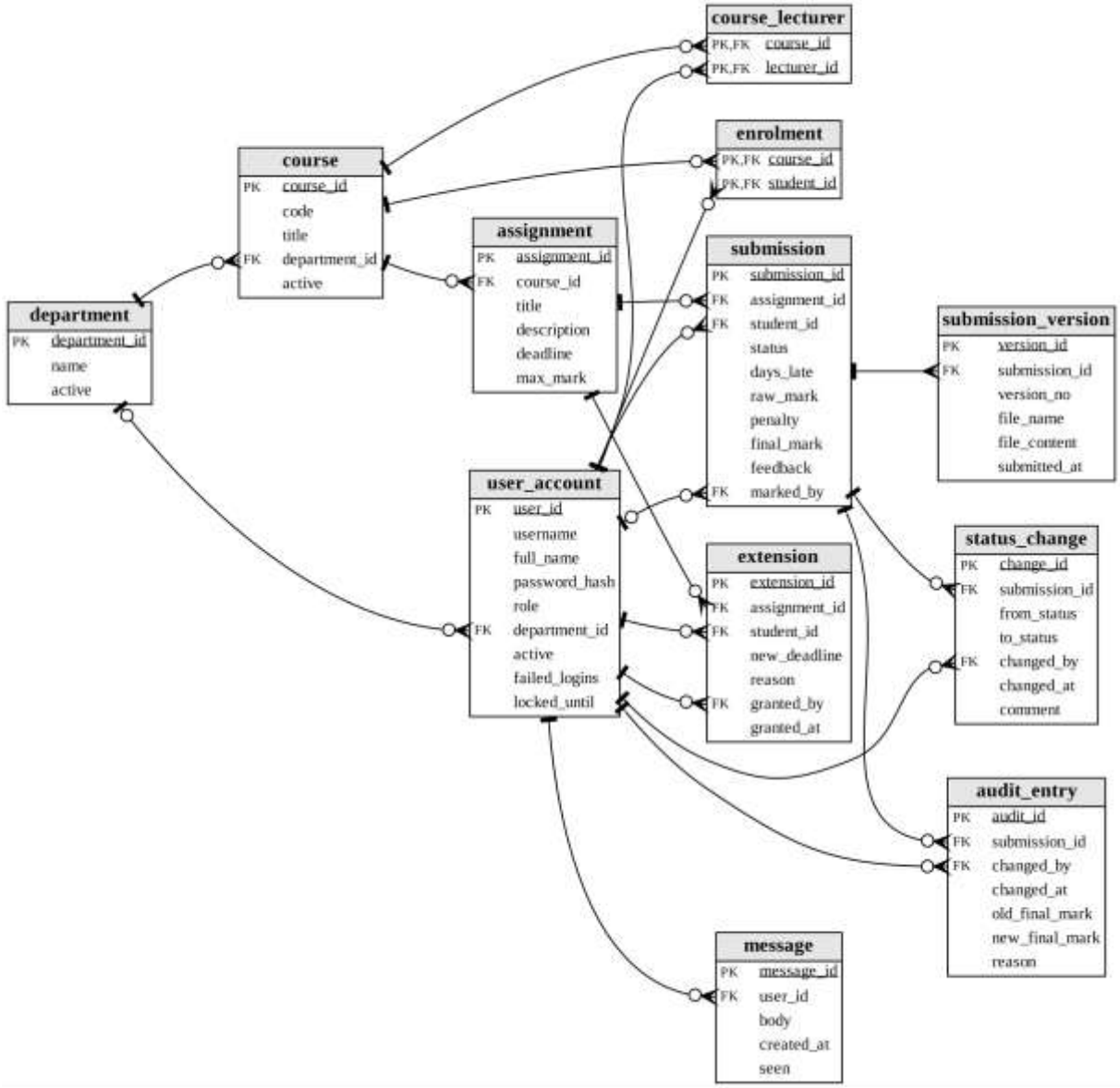


Figure 5.1: Entity-relationship diagram of the database (12 tables)

5.3 Data dictionary

Tables 5.1 to 5.12 describe every column of every table. The type is the SQLite type. Dates have the form YYYY-MM-DDTHH:MM.

Table 5.1: Table department

A department of the school. It is the class Department.

Column	Type	Null	Meaning and rules
department_id	INTEGER	NOT NULL	Primary key.
name	TEXT	NOT NULL	Name of the department. Must be unique and not blank.
active	INTEGER	NOT NULL	1 if the department is in use, 0 if it was deactivated. Default 1.

Table 5.2: Table user_account

All users of the system, of every role. It is the abstract class User and its four sub-classes (Section 5.4). The name user_account is used because USER is a reserved word in some SQL systems.

Column	Type	Null	Meaning and rules
user_id	INTEGER	NOT NULL	Primary key.
username	TEXT	NOT NULL	The name used to log in. Must be unique, also if only the capital letters differ (NOCASE), and not blank.
full_name	TEXT	NOT NULL	The name of the person. Not blank.
password_hash	TEXT	NOT NULL	The salted Argon2id hash of the password. The password itself is never saved (NFR-03).
role	TEXT	NOT NULL	One of ADMINISTRATOR, LECTURER, HEAD_OF_DEPARTMENT, STUDENT.
department_id	INTEGER	Null	Foreign key to department. It has a value if and only if the role is HEAD_OF_DEPARTMENT. A CHECK enforces this. NFR-14 allows the department only for Heads of Department.
active	INTEGER	NOT NULL	1 if the user can log in, 0 if the user was deactivated. Default 1.
failed_logins	INTEGER	NOT NULL	Number of wrong passwords in a row, from 0 to 4. At the fifth wrong password the account is locked (FR-03), and the counter goes back to 0. Default 0.
locked_until	TEXT	Null	Date and time up to which the account is locked. Empty if it is not locked.

Table 5.3: Table course

A course. It is the class Course.

Column	Type	Null	Meaning and rules
course_id	INTEGER	NOT NULL	Primary key.
code	TEXT	NOT NULL	Course code, for example SCI101. Unique and not blank.
title	TEXT	NOT NULL	Name of the course. Not blank.
department_id	INTEGER	NOT NULL	Foreign key to department: the department that owns the course. BR-07 uses it to decide which Head of Department may moderate.
active	INTEGER	NOT NULL	1 if the course is in use, 0 if deactivated. Default 1.

Table 5.4: Table course_lecturer

Link table: which lecturers teach which courses (FR-07).

Column	Type	Null	Meaning and rules
course_id	INTEGER	NOT NULL	Foreign key to course. Part of the primary key.
lecturer_id	INTEGER	NOT NULL	Foreign key to user_account. Part of the primary key. A trigger refuses a user whose role is not LECTURER.

Table 5.5: Table enrolment

Link table: which students are enrolled in which courses (FR-08).

Column	Type	Null	Meaning and rules
course_id	INTEGER	NOT NULL	Foreign key to course. Part of the primary key.

Column	Type	Null	Meaning and rules
student_id	INTEGER	NOT NULL	Foreign key to user_account. Part of the primary key. A trigger refuses a user whose role is not STUDENT.

Table 5.6: Table assignment

An assignment of a course. It is the class Assignment.

Column	Type	Null	Meaning and rules
assignment_id	INTEGER	NOT NULL	Primary key.
course_id	INTEGER	NOT NULL	Foreign key to course.
title	TEXT	NOT NULL	Title. Not blank.
description	TEXT	NOT NULL	What the students must do. Not blank.
deadline	TEXT	NOT NULL	The deadline for all students. The deadline of one student may be later because of an extension (BR-04). Must be a real date and time.
max_mark	INTEGER	NOT NULL	The maximum mark, a whole number from 1 to 100 (BR-09).

Table 5.7: Table extension

An extension of the deadline for one student and one assignment. It is the class Extension. The student's current deadline is the new_deadline of the latest row, and it is the assignment deadline if there is no row.

Column	Type	Null	Meaning and rules
extension_id	INTEGER	NOT NULL	Primary key. A bigger number is a later extension.
assignment_id	INTEGER	NOT NULL	Foreign key to assignment.
student_id	INTEGER	NOT NULL	Foreign key to user_account: the student who got the extension.
new_deadline	TEXT	NOT NULL	The new deadline of this student. The service checks that it is later than the current one (BR-04).
reason	TEXT	NOT NULL	The reason. Not blank (BR-04).
granted_by	INTEGER	NOT NULL	Foreign key to user_account: the lecturer who gave it.
granted_at	TEXT	NOT NULL	Date and time when it was given.

Table 5.8: Table submission

The work of one student for one assignment, with its status and mark. It is the class Submission. There is one row for each student and assignment, and the versions of the file are in submission_version.

Column	Type	Null	Meaning and rules
submission_id	INTEGER	NOT NULL	Primary key.
assignment_id	INTEGER	NOT NULL	Foreign key to assignment.
student_id	INTEGER	NOT NULL	Foreign key to user_account. The pair (assignment_id, student_id) is unique.
status	TEXT	NOT NULL	One of SUBMITTED, LATE, MARKED, RETURNED, MODERATED, RELEASED (Table 9 of Phase 2; Figure 4.1).
days_late	INTEGER	NOT NULL	The number n of BR-01. It is 0 if the work is on time. Default 0.

Column	Type	Null	Meaning and rules
raw_mark	REAL	Null	The mark that the lecturer typed, from 0 to 100. Empty until the work is marked.
penalty	REAL	Null	The marks taken away by BR-01. Empty until the work is marked.
final_mark	REAL	Null	The final mark, from 0 to 100. It is the raw mark minus the penalty, and it can be changed by a Head of Department (T11). Empty until the work is marked.
feedback	TEXT	Null	The written feedback. It is compulsory when the work is marked.
marked_by	INTEGER	Null	Foreign key to user_account: the lecturer who marked the work. Empty until it is marked.

Table 5.9: Table submission_version

One version of the file of a submission (UC-18 keeps the older versions). It is the class SubmissionVersion.

Column	Type	Null	Meaning and rules
version_id	INTEGER	NOT NULL	Primary key.
submission_id	INTEGER	NOT NULL	Foreign key to submission.
version_no	INTEGER	NOT NULL	1 for the first file, 2 for the next, and so on. The pair (submission_id, version_no) is unique. The latest version is the one that is marked.
file_name	TEXT	NOT NULL	The name of the file. Not blank.
file_content	BLOB	NOT NULL	The bytes of the file. The size is checked: at least 1 byte and at most 5,242,880 bytes (5 MB).
submitted_at	TEXT	NOT NULL	Date and time when this version was handed in. This is the time S of BR-01.

Table 5.10: Table status_change

The history of the status of a submission, with one row for each change (Table 9; FR-24). It is the class StatusChange.

Column	Type	Null	Meaning and rules
change_id	INTEGER	NOT NULL	Primary key.
submission_id	INTEGER	NOT NULL	Foreign key to submission.
from_status	TEXT	Null	The old status. Empty for the first row of a submission.
to_status	TEXT	NOT NULL	The new status.
changed_by	INTEGER	NOT NULL	Foreign key to user_account: the user who made the change.
changed_at	TEXT	NOT NULL	Date and time of the change.
comment	TEXT	Null	A comment, for example the comment of a Head of Department who returns a mark (T12).

Table 5.11: Table audit_entry

The audit log (BR-05, NFR-09). One row is written every time a final mark is saved or changed. Rows are never changed or deleted. It is the class AuditEntry.

Column	Type	Null	Meaning and rules
audit_id	INTEGER	NOT NULL	Primary key.
submission_id	INTEGER	NOT NULL	Foreign key to submission.
changed_by	INTEGER	NOT NULL	Foreign key to user_account: the lecturer or Head of Department who made the change.
changed_at	TEXT	NOT NULL	Date and time of the change.
old_final_mark	REAL	Null	The final mark before the change. Empty for the first mark.
new_final_mark	REAL	NOT NULL	The final mark after the change, from 0 to 100.
reason	TEXT	NOT NULL	The reason. The first mark has the reason "initial marking". Not blank.

Table 5.12: Table message

A message that is shown to a user at the next log in (FR-23). It is the class Message.

Column	Type	Null	Meaning and rules
message_id	INTEGER	NOT NULL	Primary key.
user_id	INTEGER	NOT NULL	Foreign key to user_account: the user who gets the message.
body	TEXT	NOT NULL	The text. Not blank.
created_at	TEXT	NOT NULL	Date and time when the message was made.
seen	INTEGER	NOT NULL	0 until the user has seen it, then 1. Default 0.

5.4 From classes to tables

Table 5.13 shows which table holds each class of Section 3. Classes that only do work and keep no data (the services, menus, PenaltyStrategy, Clock, PasswordHasher, UserFactory, the exceptions and the repositories) have no table. The tables differ from the classes in these points:

- The four user classes are one table, because a relational database has no inheritance. Table 5.14 gives the choice.
- Each many-to-many association is a link table, and each enumeration is a TEXT column with a CHECK.
- Function and permissions() are not stored, because the rights of a role are fixed in the program. Operations such as visibleStatus() are not stored either.
- days_late, penalty, final_mark and status are stored although they can be worked out. Table 5.16 gives the reasons.

Table 5.13: Class to table

Class (Section 3)	Table	Notes
User (abstract), Administrator, Lecturer, HeadOfDepartment, Student	user_account	One table for the whole family. The class is told by the column role. The enumeration Role is the column role. Table 5.14 gives the reason.

Class (Section 3)	Table	Notes
Department	department	Direct. HeadOfDepartment.departmentId is user_account.department_id.
Course	course	Direct.
Course and Lecturer (teaches)	course_lecturer	Many-to-many association, so a link table.
Course and Student (enrolled in)	enrolment	Many-to-many association, so a link table.
Assignment	assignment	Direct.
Extension	extension	Direct.
Submission	submission	Direct. The enumeration SubmissionStatus is the column status. visibleStatus() is not stored. It is worked out from status.
SubmissionVersion	submission_version	Direct. The attribute content (vector of bytes) is the column file_content.
StatusChange	status_change	Direct.
AuditEntry	audit_entry	Direct.
Message	message	Direct.
Function and permissions()	none	The rights of each role are fixed in the program (Table 10 of Phase 2). They are not saved, because the administrator cannot change them.

A relational database has no inheritance, so there are three well-known ways to store the user classes (Fowler, 2002). Table 5.14 compares them. One table with a role column was chosen.

Table 5.14: Ways to store the user classes

Way	What it means here	Judgement
One table for the family (chosen)	One table user_account with a column role. The only attribute that a sub-class adds is the department of a Head of Department, which is one column that may be empty.	Simple. A log in reads one row, with no join. A foreign key such as submission.student_id points to one table. The weak point is the empty column, which a CHECK controls (department_id is filled if and only if the role is HEAD_OF_DEPARTMENT).
One table for each class	A table user_account with the common columns, and four more tables, one for each role, joined by the key.	The four sub-tables would have almost no columns (three of them none). Every log in would need a join. Too much structure for no gain.
One table for each concrete class	Four tables, each with all the common columns repeated.	The common columns are repeated four times. A foreign key like submission.student_id cannot point to a student table and a lecturer table at the same time, so granted_by and marked_by could not be checked by the database.

5.5 Normalization

Each table was checked against the three normal forms. 1NF means one value in each cell and no repeating group. 2NF means that every column depends on the whole key. 3NF means that no column depends on a column that is not the key. Table 5.15 gives the result. A few columns

save a value that could be worked out from other columns. This breaks 3NF on purpose, and Table 5.16 gives the reason for each.

Table 5.15: Normal form check

Table	Key	Result
department	department_id	3NF. Name and active depend only on the department.
user_account	user_id	3NF. All columns describe the user. department_id depends on the user and is allowed only for one role, which a CHECK controls.
course	course_id	3NF. The department is saved as a key, not as a name, so the name of a department is saved once.
course_lecturer, enrolment	(course_id, lecturer_id) and (course_id, student_id)	3NF. They have no columns outside the key. They remove the many-to-many repeating groups: without them a course would need a list of lecturers and a list of students in one cell, which breaks 1NF.
assignment	assignment_id	3NF. Deadline and max_mark describe the assignment, not the course.
extension	extension_id	3NF. Each row is one extension. The current deadline is not saved as a second copy in assignment or submission.
submission	submission_id	3NF, except for the deliberate redundancy in Table 5.16.
submission_version	version_id	3NF. The versions are a repeating group of a submission, so they have their own table.
status_change	change_id	3NF, except for the note in Table 5.16.
audit_entry	audit_id	3NF, except for the note in Table 5.16.
message	message_id	3NF.

Table 5.16: Deliberate redundancy

Column	Can be worked out from	Why it is saved	What keeps it correct
submission.days_late	The time S of the latest version, and the student's deadline D (BR-01)	The list of 800 submissions shows it for every row. An extension changes D later (T4, T5), so the number is a fact at one moment.	The service writes it in the same transaction as the version or the extension (AR-5).
submission.penalty	raw_mark, max_mark and days_late (BR-01)	A mark that was shown to a lecturer must not change if a rule is changed later. The audit log also compares final marks.	MarkingService::saveMark works out and saves it in one transaction.
submission.final_mark	raw_mark minus penalty	After a Head of Department adjusts the mark (T11), the final mark is not raw mark minus penalty any more. It is then a fact of its own, so it cannot be worked out.	A CHECK allows it only when the work is marked. The audit entry records every change.
submission.status	to_status of the latest status_change	Every list asks for a status (FR-20, FR-22, FR-25). Reading it from the history would need a sub-query on each row.	Submission::changeStatus is the only way to change it. The service writes the status and the status_change row in one transaction.

Column	Can be worked out from	Why it is saved	What keeps it correct
audit_entry.old_final_mark	new_final_mark of the previous entry	An audit entry must be complete when it is read alone, because it is never changed.	AuditService::record reads the old value in the same transaction.
status_change.from_status	to_status of the previous row	The same reason as above.	The same service call writes both.

6. Requirements traceability matrix

Table 6.1 is version 2 of the traceability matrix of Phase 2. It links each functional requirement to its use case and now also to its design elements: the classes and operations of Section 3, the figures of Sections 3 and 4, and the tables of Section 5. The source files and the test cases are added in the later phases.

Table 6.1: Requirements traceability matrix, version 2 (design elements)

FR	Requirement	Use case	Classes and operations (Section 3)	Figures	Tables (Section 5)
FR-01	Log in	UC-01	AuthService.login; User.isLocked; PasswordHasher.verify; IUserRepository.findByUsername	3.1, 3.4, 3.7, 3.9	user_account
FR-02	Who can do what	All use cases	User.can and permissions() of each User subclass; the enumeration Function; the check at the start of every service operation	3.1, 3.4 to 3.6	user_account (role)
FR-03	Account lock	UC-01	AuthService.login (MAX_WRONG, LOCK_MINUTES); User.failedLogins, lockedUntil and isLocked; IUserRepository.update; Clock.now	3.1, 3.4, 3.7	user_account (failed_logins, locked_until)
FR-04	Manage user accounts	UC-02	AdminService.createUser, editUser, deactivateUser; UserFactory.create; IUserRepository.add, update, activeHeadOf	3.1, 3.4, 3.8, 3.9	user_account
FR-05	Set new password	UC-02	AdminService.resetPassword; PasswordHasher.hash; IUserRepository.update	3.4, 3.7	user_account (password_hash)
FR-06	Departments and courses	UC-03	AdminService.createDepartment, editDepartment, deactivateDepartment, createCourse, editCourse, deactivateCourse; ICourseRepository; Department; Course	3.2, 3.4, 3.9	department, course
FR-07	Assign lecturers	UC-04	AdminService.assignLecturer; ICourseRepository.assignLecturer	3.2, 3.4, 3.9	course_lecturer
FR-08	Enroll students	UC-05	AdminService.enrolStudent, removeStudent; ICourseRepository.enrol, removeEnrolment	3.2, 3.4, 3.9	enrolment
FR-09	Create assignment	UC-06	AssignmentService.createAssignment; IAssignmentRepository.add; Assignment	3.2, 3.5, 3.10	assignment
FR-10	Individual extension	UC-07	AssignmentService.grantExtension; Extension; IAssignmentRepository.addExtension, latestExtension; ISubmissionRepository.addStatusChange (when a Late submission becomes Submitted)	3.2, 3.5, 3.10, 4.1	extension
FR-11	View submissions	UC-08	MarkingService.listSubmissions, exportFile; ISubmissionRepository.listByAssignment, latestVersion; InputReader.writeFileBytes	3.3, 3.6, 3.10	submission, submission_version
FR-12	Mark and feedback	UC-08, UC-09	MarkingService.saveMark; Submission.changeStatus; SubmissionRow	3.3, 3.6, 4.1, 4.2	submission
FR-13	Late penalty	UC-08, UC-10	MarkingService.previewMark; PenaltyStrategy.daysLate, apply; AssignmentService.deadlineFor; PenaltyResult; MarkPreview	3.3, 3.6, 3.7, 4.2, 4.4	submission (days_late, penalty, final_mark)
FR-14	Audit log	UC-08, UC-11, UC-13	AuditService.record, called by MarkingService.saveMark and ModerationService.adjust inside their transaction; IAuditRepository.add; AuditEntry	3.3, 3.5, 3.6, 4.6	audit_entry
FR-15	View courses and assignments	UC-15	SubmissionService.listMyAssignments; AssignmentService.listMyCourses, deadlineFor; Submission.visibleStatus; StudentAssignmentRow	3.2, 3.6, 4.3	course, assignment, enrolment, submission

FR	Requirement	Use case	Classes and operations (Section 3)	Figures	Tables (Section 5)
FR-16	Submit assignment	UC-16	SubmissionService.prepareSubmission, confirmSubmission (MAX_BYTES); InputReader.readFileBytes; ISubmissionRepository.add, addVersion; SubmissionVersion; Receipt	3.3, 3.6, 4.2 to 4.5	submission, submission_version, status_change
FR-17	Resubmit	UC-18	SubmissionService.prepareSubmission, confirmSubmission; Submission.canBeReplaced; ISubmissionRepository.addVersion, latestVersion	3.3, 3.6, 4.1, 4.4, 4.5	submission_version
FR-18	Late submission	UC-17	SubmissionService.prepareSubmission (fills SubmissionPreview); PenaltyStrategy.daysLate, apply; Submission.status LATE	3.6, 3.7, 4.1, 4.4, 4.5	submission (status, days_late)
FR-19	View released marks	UC-19	SubmissionService.listMyResults; Submission.isVisibleToStudent; ResultRow	3.3, 3.6, 4.1, 4.2	submission
FR-20	HoD views marks	UC-12	ModerationService.listMarked; ISubmissionRepository.listByStatus	3.6, 3.10, 4.6	submission, course
FR-21	Moderate marks	UC-12, UC-13	ModerationService.approve, adjust, returnToLecturer; Submission.changeStatus; AuditService.record; MessageService.notify	3.6, 4.1, 4.6	submission, status_change, audit_entry, message
FR-22	Release marks	UC-14	ModerationService.listModerated, release, releaseAssignment; Submission.changeStatus; MessageService.notify	3.6, 4.1, 4.7	submission, status_change, message
FR-23	Messages	UC-12, UC-14	MessageService.notify, takeWaiting; IMessageRepository.add, listUnseen, markSeen; Message; LoginMenu shows them	3.1, 3.5, 4.6, 4.7	message
FR-24	Submission status	UC-08, UC-12, UC-14, UC-16	Submission.changeStatus (the only way to change a status, by Table 9); SubmissionStatus; StatusChange; ISubmissionRepository.addStatusChange	3.3, 4.1	submission (status), status_change
FR-25	Assignment summary	UC-12	ModerationService.summary; ISubmissionRepository.countByStatus; AssignmentSummary	3.3, 3.6, 4.6	submission
FR-26	View audit log	UC-20	AuditService.listForMyDepartment; IAuditRepository.listByDepartment; AuditEntry	3.3, 3.5, 3.9	audit_entry

Appendix A: SQL schema

This is the complete schema for SQLite. It creates the 12 tables, the triggers and the indexes. The program runs it once, when the database is made.

```
-- Moderated LMS: database schema for SQLite (version 1.0)
-- Run "PRAGMA foreign_keys = ON;" on every new connection.
-- Dates and times are text in the form YYYY-MM-DDTHH:MM (local time, to the minute).

PRAGMA foreign_keys = ON;

CREATE TABLE department (
    department_id  INTEGER PRIMARY KEY,
    name           TEXT    NOT NULL UNIQUE CHECK (length(trim(name)) > 0),
    active         INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0, 1))
);

CREATE TABLE user_account (
    user_id        INTEGER PRIMARY KEY,
    username       TEXT    NOT NULL UNIQUE COLLATE NOCASE CHECK (length(trim(username)) > 0),
    full_name      TEXT    NOT NULL CHECK (length(trim(full_name)) > 0),
    password_hash  TEXT    NOT NULL CHECK (length(password_hash) > 0),
    role           TEXT    NOT NULL CHECK (role IN ('ADMINISTRATOR', 'LECTURER',
'HEAD_OF_DEPARTMENT', 'STUDENT')),
    department_id  INTEGER REFERENCES department (department_id),
    active         INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0, 1)),
    failed_logins  INTEGER NOT NULL DEFAULT 0 CHECK (failed_logins BETWEEN 0 AND 4),
    locked_until   TEXT    CHECK (locked_until IS NULL OR strftime('%Y-%m-%dT%H:%M', locked_until)
IS locked_until),
    CHECK ((role = 'HEAD_OF_DEPARTMENT') = (department_id IS NOT NULL))
);

-- A department can have only one active Head of Department.
CREATE UNIQUE INDEX one_active_hod_per_department
    ON user_account (department_id)
    WHERE role = 'HEAD_OF_DEPARTMENT' AND active = 1;

CREATE TABLE course (
    course_id      INTEGER PRIMARY KEY,
    code           TEXT    NOT NULL UNIQUE CHECK (length(trim(code)) > 0),
    title          TEXT    NOT NULL CHECK (length(trim(title)) > 0),
    department_id  INTEGER NOT NULL REFERENCES department (department_id),
    active         INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0, 1))
);

CREATE TABLE course_lecturer (
    course_id      INTEGER NOT NULL REFERENCES course (course_id),
    lecturer_id    INTEGER NOT NULL REFERENCES user_account (user_id),
    PRIMARY KEY (course_id, lecturer_id)
);

CREATE TABLE enrolment (
    course_id      INTEGER NOT NULL REFERENCES course (course_id),
    student_id     INTEGER NOT NULL REFERENCES user_account (user_id),
    PRIMARY KEY (course_id, student_id)
);

CREATE TRIGGER course_lecturer_role_check
BEFORE INSERT ON course_lecturer
WHEN (SELECT role FROM user_account WHERE user_id = NEW.lecturer_id) <> 'LECTURER'
BEGIN
    SELECT RAISE(ABORT, 'Only a lecturer can be assigned to a course');
END;

CREATE TRIGGER enrolment_role_check
BEFORE INSERT ON enrolment
WHEN (SELECT role FROM user_account WHERE user_id = NEW.student_id) <> 'STUDENT'
BEGIN
    SELECT RAISE(ABORT, 'Only a student can be enrolled in a course');
END;

CREATE TABLE assignment (
    assignment_id  INTEGER PRIMARY KEY,
    course_id      INTEGER NOT NULL REFERENCES course (course_id),
    title          TEXT    NOT NULL CHECK (length(trim(title)) > 0),
    description    TEXT    NOT NULL CHECK (length(trim(description)) > 0),
    deadline       TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', deadline) IS deadline),
    max_mark       INTEGER NOT NULL CHECK (max_mark BETWEEN 1 AND 100)
);
```

```
CREATE TABLE extension (
    extension_id   INTEGER PRIMARY KEY,
    assignment_id  INTEGER NOT NULL REFERENCES assignment (assignment_id),
    student_id     INTEGER NOT NULL REFERENCES user_account (user_id),
    new_deadline   TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', new_deadline) IS
new_deadline),
    reason         TEXT    NOT NULL CHECK (length(trim(reason)) > 0),
    granted_by     INTEGER NOT NULL REFERENCES user_account (user_id),
    granted_at     TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', granted_at) IS granted_at)
);

CREATE TABLE submission (
    submission_id  INTEGER PRIMARY KEY,
    assignment_id  INTEGER NOT NULL REFERENCES assignment (assignment_id),
    student_id     INTEGER NOT NULL REFERENCES user_account (user_id),
    status         TEXT    NOT NULL CHECK (status IN ('SUBMITTED', 'LATE', 'MARKED', 'RETURNED',
'MODERATED', 'RELEASED')),
    days_late      INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
    raw_mark       REAL    CHECK (raw_mark BETWEEN 0 AND 100),
    penalty        REAL    CHECK (penalty >= 0),
    final_mark     REAL    CHECK (final_mark BETWEEN 0 AND 100),
    feedback       TEXT,
    marked_by      INTEGER REFERENCES user_account (user_id),
    UNIQUE (assignment_id, student_id),
    CHECK (
        (status IN ('SUBMITTED', 'LATE')
            AND raw_mark IS NULL AND penalty IS NULL AND final_mark IS NULL
            AND feedback IS NULL AND marked_by IS NULL)
        OR
        (status NOT IN ('SUBMITTED', 'LATE')
            AND raw_mark IS NOT NULL AND penalty IS NOT NULL AND final_mark IS NOT NULL
            AND feedback IS NOT NULL AND length(trim(feedback)) > 0 AND marked_by IS NOT NULL)
    )
);

CREATE TABLE submission_version (
    version_id     INTEGER PRIMARY KEY,
    submission_id  INTEGER NOT NULL REFERENCES submission (submission_id),
    version_no     INTEGER NOT NULL CHECK (version_no >= 1),
    file_name      TEXT    NOT NULL CHECK (length(trim(file_name)) > 0),
    file_content   BLOB    NOT NULL CHECK (length(file_content) BETWEEN 1 AND 5242880),
    submitted_at   TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', submitted_at) IS
submitted_at),
    UNIQUE (submission_id, version_no)
);

CREATE TABLE status_change (
    change_id      INTEGER PRIMARY KEY,
    submission_id  INTEGER NOT NULL REFERENCES submission (submission_id),
    from_status    TEXT    CHECK (from_status IS NULL OR from_status IN ('SUBMITTED', 'LATE',
'MARKED', 'RETURNED', 'MODERATED', 'RELEASED')),
    to_status      TEXT    NOT NULL CHECK (to_status IN ('SUBMITTED', 'LATE', 'MARKED', 'RETURNED',
'MODERATED', 'RELEASED')),
    changed_by     INTEGER NOT NULL REFERENCES user_account (user_id),
    changed_at     TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', changed_at) IS changed_at),
    comment        TEXT
);

CREATE TABLE audit_entry (
    audit_id        INTEGER PRIMARY KEY,
    submission_id   INTEGER NOT NULL REFERENCES submission (submission_id),
    changed_by      INTEGER NOT NULL REFERENCES user_account (user_id),
    changed_at      TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', changed_at) IS changed_at),
    old_final_mark  REAL    CHECK (old_final_mark BETWEEN 0 AND 100),
    new_final_mark  REAL    NOT NULL CHECK (new_final_mark BETWEEN 0 AND 100),
    reason          TEXT    NOT NULL CHECK (length(trim(reason)) > 0)
);

-- Audit log entries can never be changed or deleted (NFR-09).
CREATE TRIGGER audit_entry_no_update
BEFORE UPDATE ON audit_entry
BEGIN
    SELECT RAISE(ABORT, 'Audit log entries cannot be changed');
END;

CREATE TRIGGER audit_entry_no_delete
BEFORE DELETE ON audit_entry
BEGIN
    SELECT RAISE(ABORT, 'Audit log entries cannot be deleted');
END;

CREATE TABLE message (
    message_id     INTEGER PRIMARY KEY,
```

```
    user_id        INTEGER NOT NULL REFERENCES user_account (user_id),
    body           TEXT    NOT NULL CHECK (length(trim(body)) > 0),
    created_at     TEXT    NOT NULL CHECK (strftime('%Y-%m-%dT%H:%M', created_at) IS created_at),
    seen           INTEGER NOT NULL DEFAULT 0 CHECK (seen IN (0, 1))
);

CREATE INDEX idx_course_department       ON course (department_id);
CREATE INDEX idx_course_lecturer_lect    ON course_lecturer (lecturer_id);
CREATE INDEX idx_enrolment_student       ON enrolment (student_id);
CREATE INDEX idx_assignment_course       ON assignment (course_id);
CREATE INDEX idx_extension_lookup        ON extension (assignment_id, student_id);
CREATE INDEX idx_submission_assign_stat  ON submission (assignment_id, status);
CREATE INDEX idx_submission_student      ON submission (student_id);
CREATE INDEX idx_status_change_submission ON status_change (submission_id);
CREATE INDEX idx_audit_submission        ON audit_entry (submission_id);
CREATE INDEX idx_message_user_seen       ON message (user_id, seen);
```

References

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. and Stal, M. (1996) Pattern-Oriented Software Architecture, Volume 1: A System of Patterns. Chichester: Wiley.

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley.

Krasner, G.E. and Pope, S.T. (1988) A cookbook for using the Model-View-Controller user interface paradigm in Smalltalk-80. Journal of Object-Oriented Programming, 1(3), pp. 26-49.

Martin, R.C. (2003) Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River: Prentice Hall.

Object Management Group (2017) Unified Modeling Language, Version 2.5.1. Available at: https://www.omg.org/spec/UML/2.5.1 (Accessed: 19 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.